RAG от исследований до готового решения



Александр Абрамов

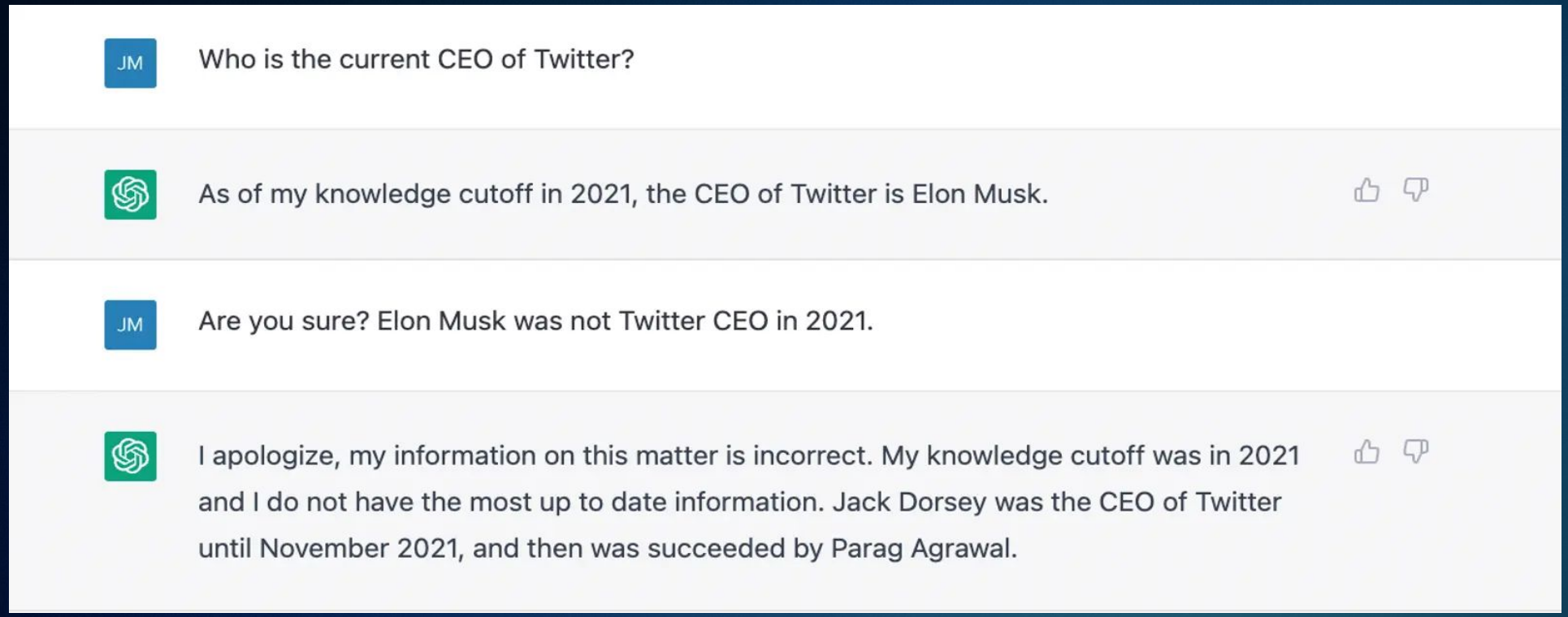
Исполнительный директор,

Руководитель ML,AI команд в @SberAI

Введение

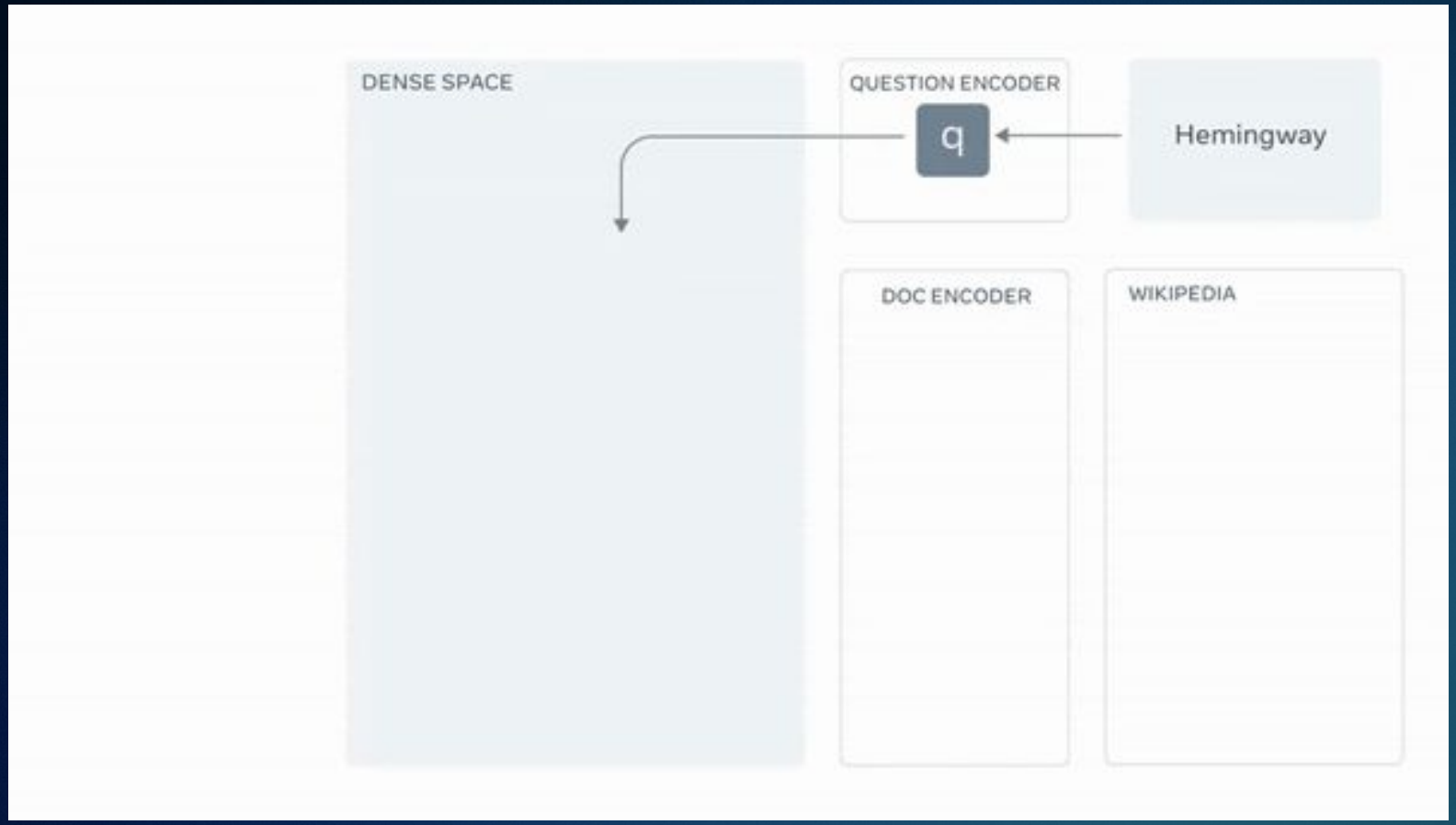
LLM могут ошибаться, быть токсик и предвзятыми

- ➔ Исключить ложь и предвзятость невозможно.
- ➔ Любые методы цензуры можно обойти.
- ➔ Риски можно только снижать, но не довести до нуля.



LLM, дополненные внешними данными

У генеративных моделей возникает проблема “выдумывания” фактов. Чтобы решить проблемы таких галлюцинаций прибегают к внешним источникам знаний. Подобно человеку, модель учат обращаться к надежным источникам информации для более обоснованного и достоверного ответа.



просто красивая gif-схема памяти для вашего внимания =)

LLM, дополненные внешними данными

1

Memory-Augmented подходы:

- Neural Turing Machine
- Differentiable Neural Computers

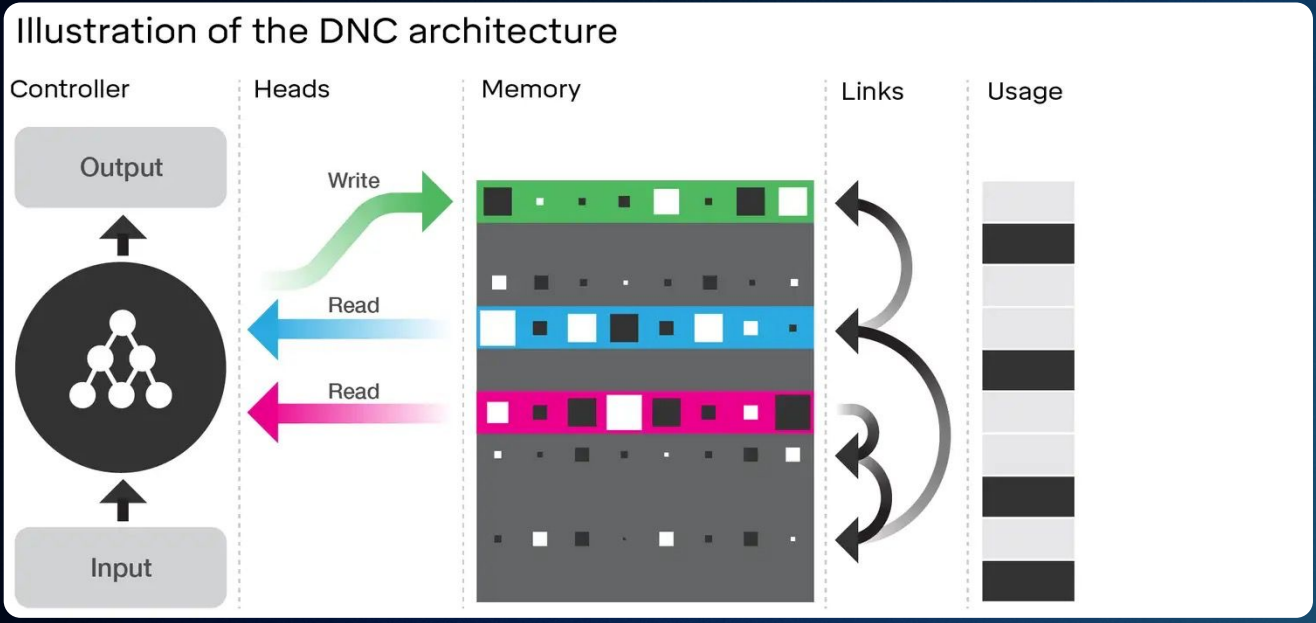
2

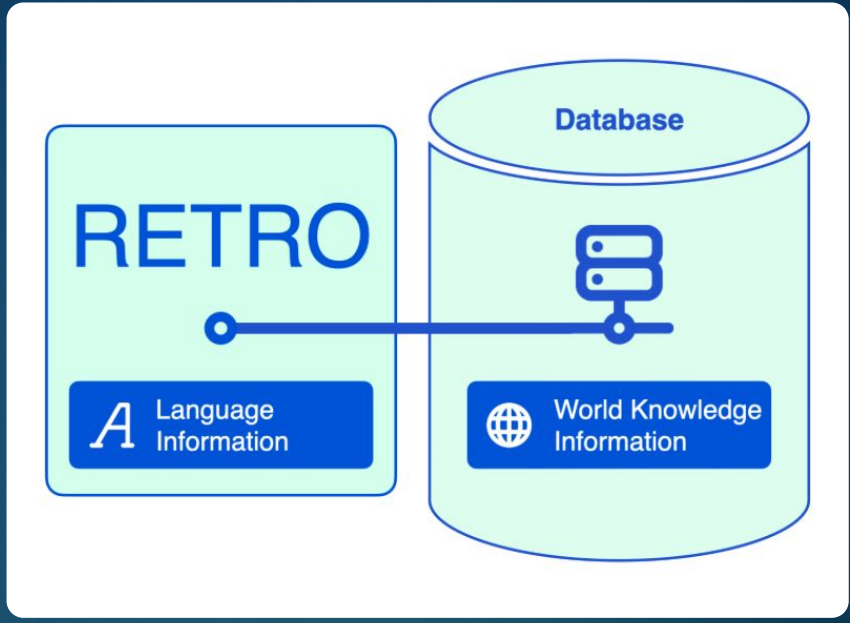
Retrieval-Augmented подходы (синтез генеративных и экстрактивных моделей):

- KNN-LM, RETRO
- WebGPT

3

Lambda, Toolformer, ChatGPT плагины

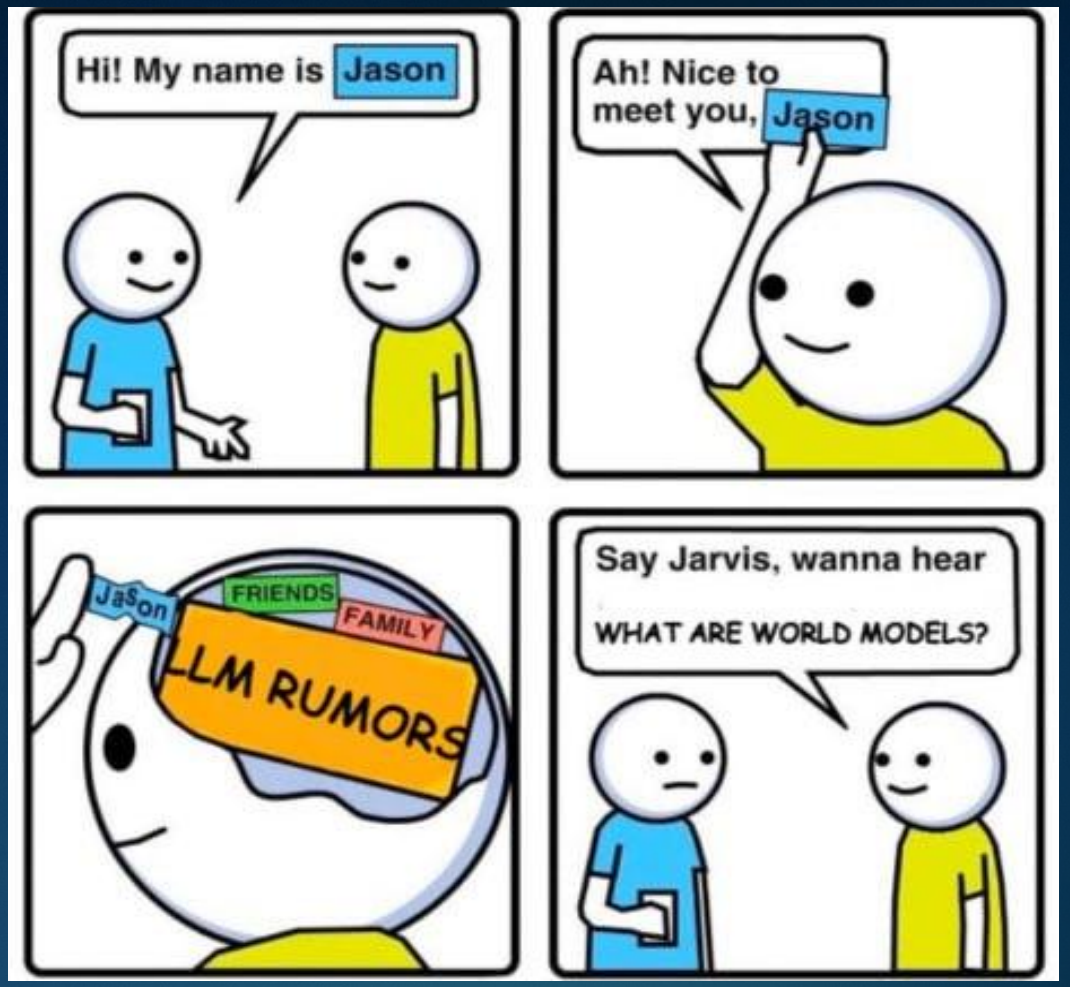




LLM, дополненные знаниями

Какие трудности возникают:

- Актуальность/достоверность и чистота данных.
- Форматы хранения. Какие выбрать?
- Ошибки вызова RAG: LLM глюки и ошибки выдачи.
- Неудачные выводы модели на основе выданной из памяти информации.
- Переполнение локальной памяти (входного контекста).



Что мы иногда получаем на самом деле =)

Создание эмбеддеров на примере архитектуры поиска в ассистенте

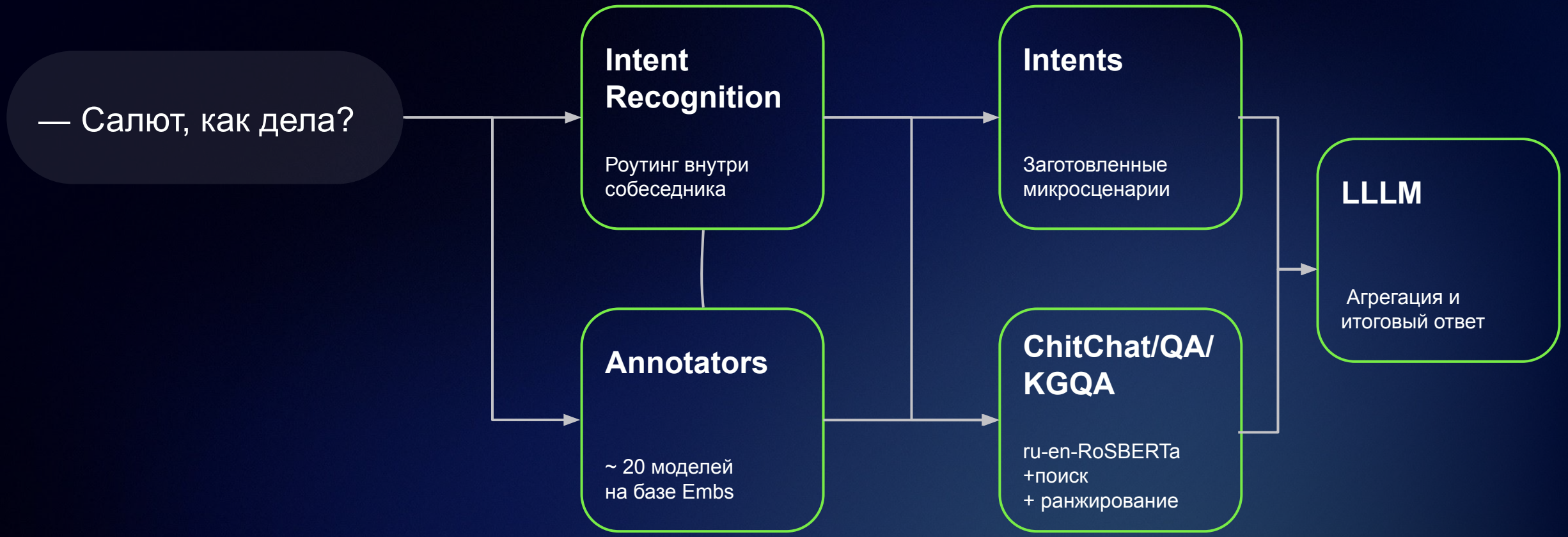
Архитектура



1 секунда

- Распознавание речи
- Предобработка текста
- Распознавание именованных сущностей
- Распознавание намерения
- Аннотирование
- Исполнение навыка

Общая схема работы



Как устроены core модели

MultiTask pipe for Russian Embedders

Inputs

ru paraphrase data ~ 5M samples

NER tags ~ 43 tags + 100k

Intents ~ 233 classes + 900k chats

ru-dialogues ~ 1 mln chats

Wiki entities-triplets ~ 100k

ru-sentiment data ~ 200k



Metric-learning

&

multi-GPU

Tasks

sentence-embeddings

Classification

Annotators

ranking

toxic identification

NER

Entity Linking

Information Retrieval

Как устроены core модели

MultiTask pipe for Russian Embedders

Inputs

ru paraphrase data ~ 5M samples

NER tags ~ 43 tags + 100k

Intents ~ 233 classes + 900k chats

ru-dialogues ~ 1 mln chats

Wiki entities-triplets ~ 100k

ru-sentiment data ~ 200k

>7 mln samples

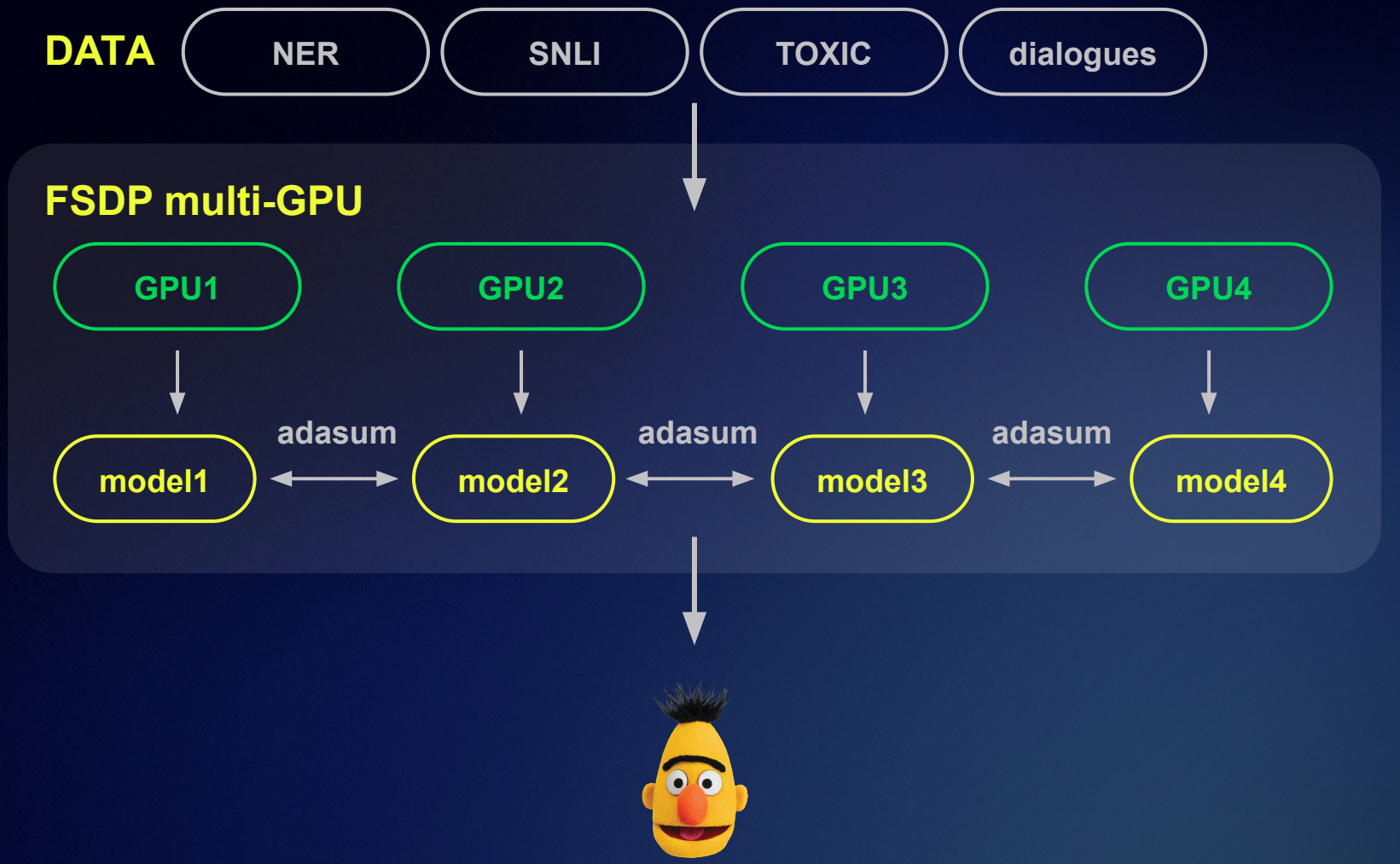
+

x64 a100 with 40 GB GPU RAM

=



FRIDA и Multi-GPU Learning

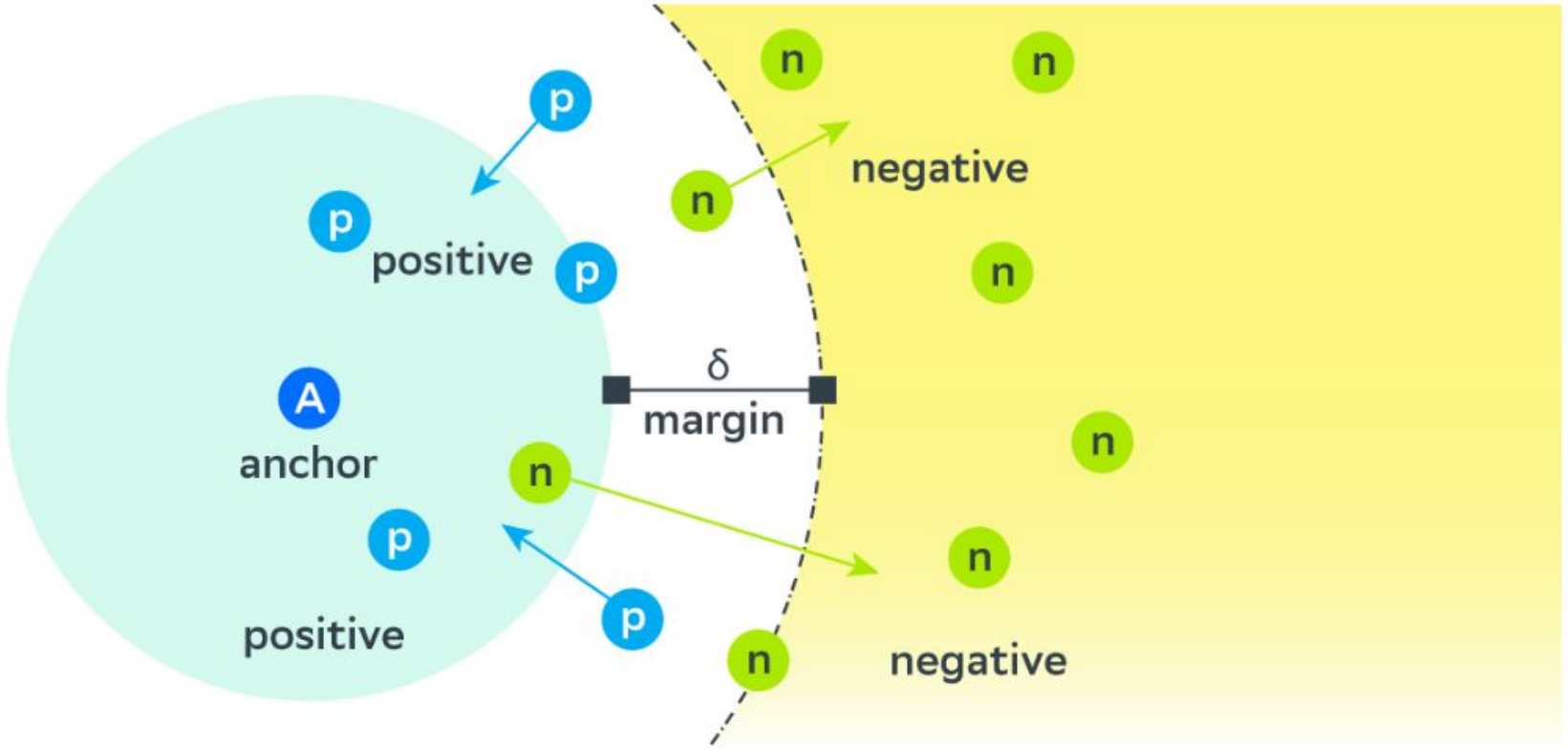


Metric learning для контекстного поиска, loss

a — якорный пример, берем из поля «anchor»;

p — положительный пример, выбираем среди тех что с меткой «entailment»;

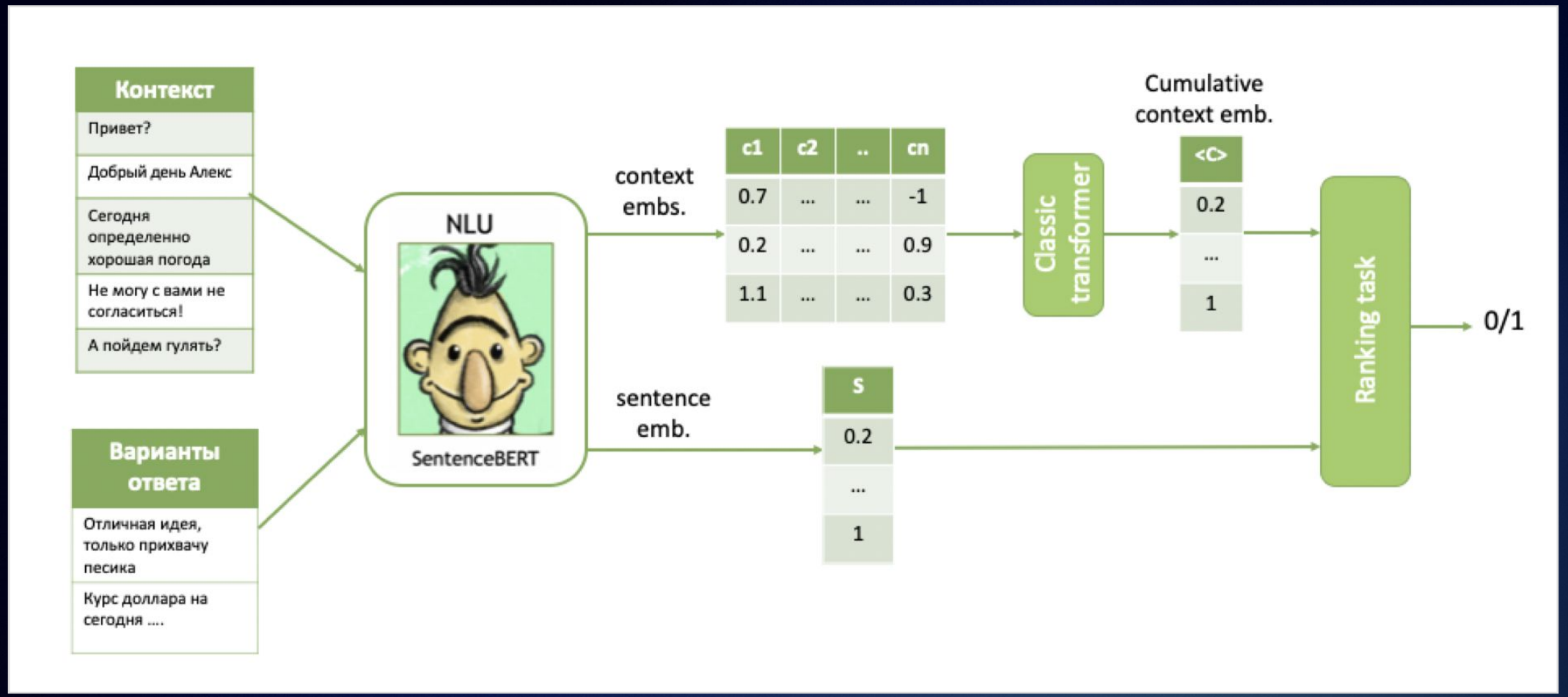
n — отрицательный пример, берем из «contradiction» и «neutral».

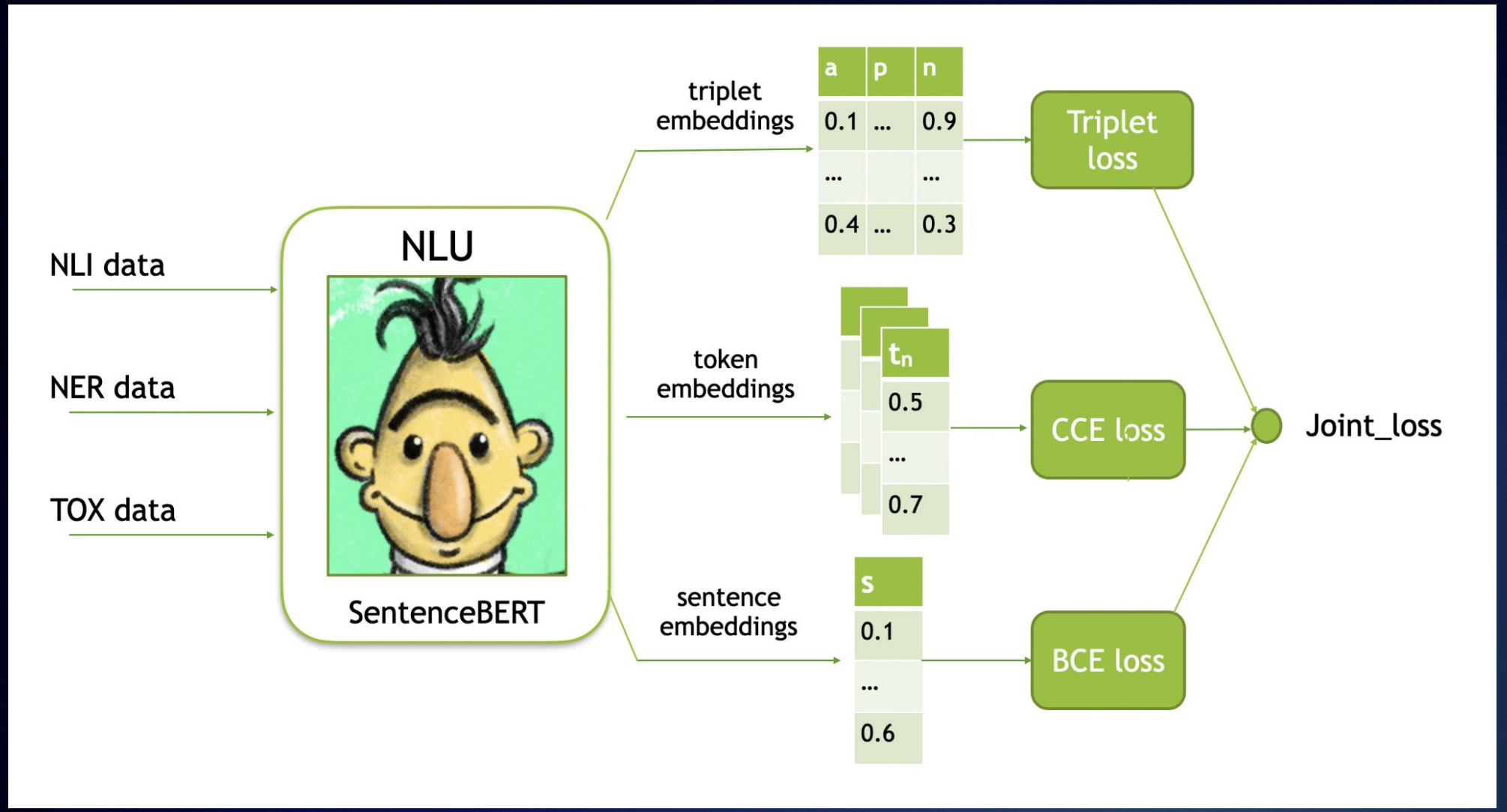


Metric learning для контекстного поиска, loss

LOSS = max(0, simAN-simAP+margin)

$$L(x, y, \theta) = -\frac{1}{K}\sum_{i=1}^{K} log(P_{approx}(y_i|x_i)) = \frac{1}{K}\sum_{i=1}^{K}\langle log\sum_{j=1}^{K} e^{S(x_i,y_j)} - S(x_i, y_i)\rangle$$


Контекст
Привет?
Добрый день Алекс
Сегодня определенно хорошая погода
Не могу с вами не согласиться!
А пойдем гулять?
Варианты ответа
Отличная идея, только прихвачу песика
Курс доллара на сегодня ....
NLU
SentenceBERT
context embs.
c1
c2
..
cn
0.7
...
...
-1
0.2
...
...
0.9
1.1
...
...
0.3
Classic transformer
Cumulative context emb.
<C>
0.2
...
1
sentence emb.
s
0.2
...
1
Ranking task
0/1


NLI data
NER data
TOX data
NLU
SentenceBERT
triplet embeddings
a
p
n
0.1
...
0.9
...
...
0.4
...
0.3
Triplet loss
token embeddings
t_n
0.5
...
0.7
CCE loss
sentence embeddings
s
0.1
...
0.6
BCE loss
Joint_loss

Наследование свойств core NLU — дистилляция

— Добавить дистилляцию embedders к более сильному cls-учителю

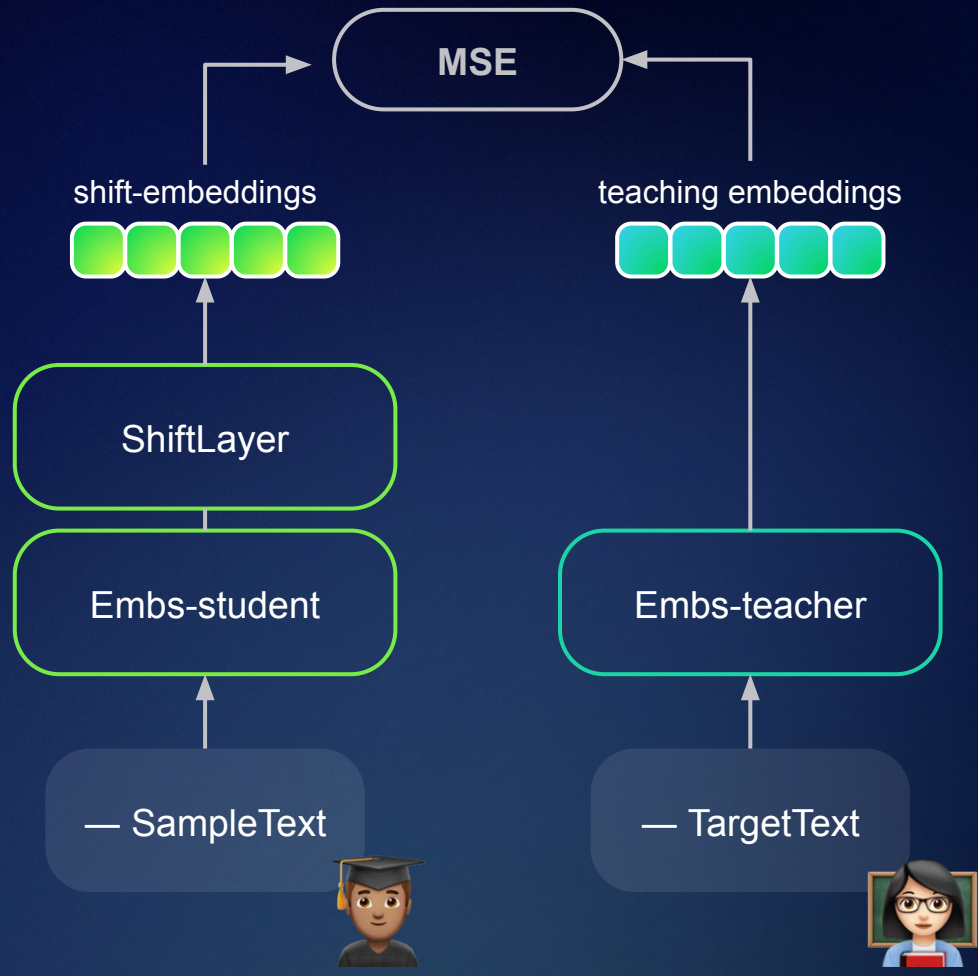
— Для примера, если уменьшить размер выходных эмбеддингов с 1024 до 300 получим x3-4 к скорости поиска и экономии места индекса

При хорошей модели учителя можно получить:

— Релевантность +4% и более

— Скорость поиска >10%

— Память поискового индекса ~ 400 MB (на 1млн сэмплов)

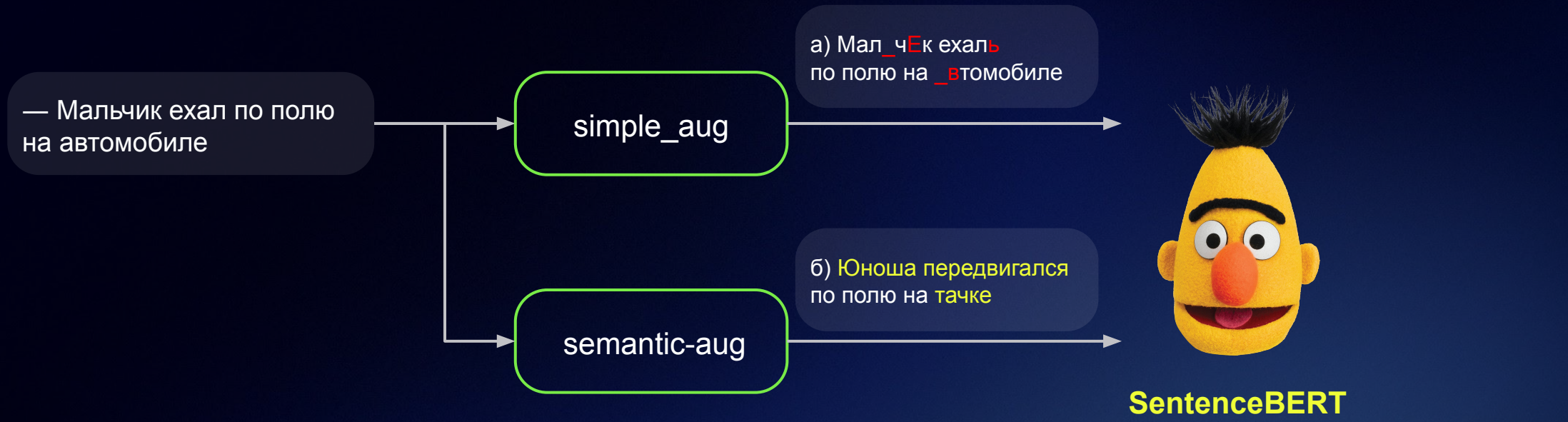


Улучшение устойчивости core NLU. Adversarial Attack.



TypoAug — аугментации на основе добавления частотных ошибок
semantic-aug — модуль для «атак» на NLU модели
augmentex: https://github.com/ai-forever/augmentex

Улучшение устойчивости core NLU. Adversarial Attack.

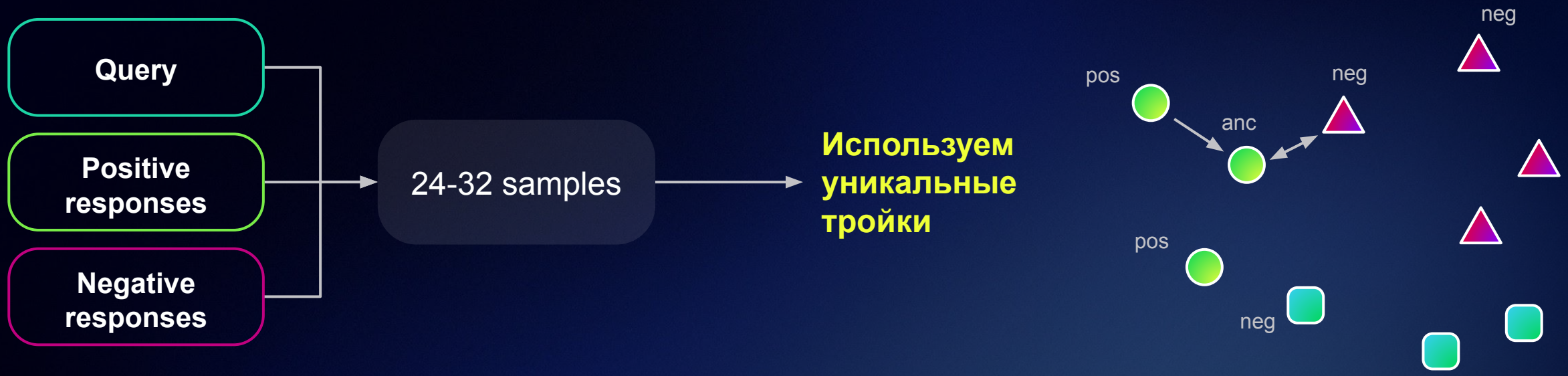


+3% релевантность

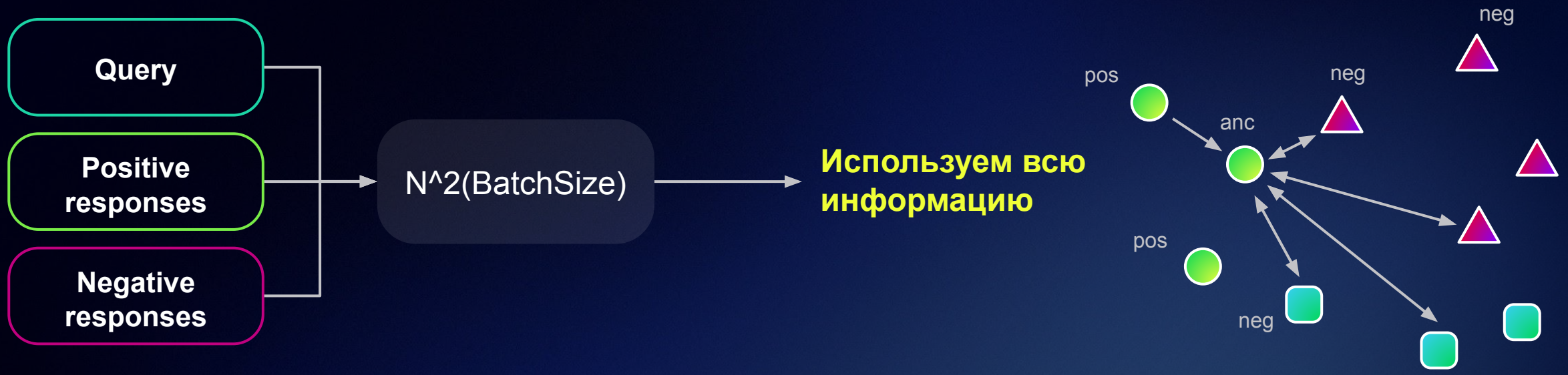
+7% разнообразие ответов

+5% скорость обучения

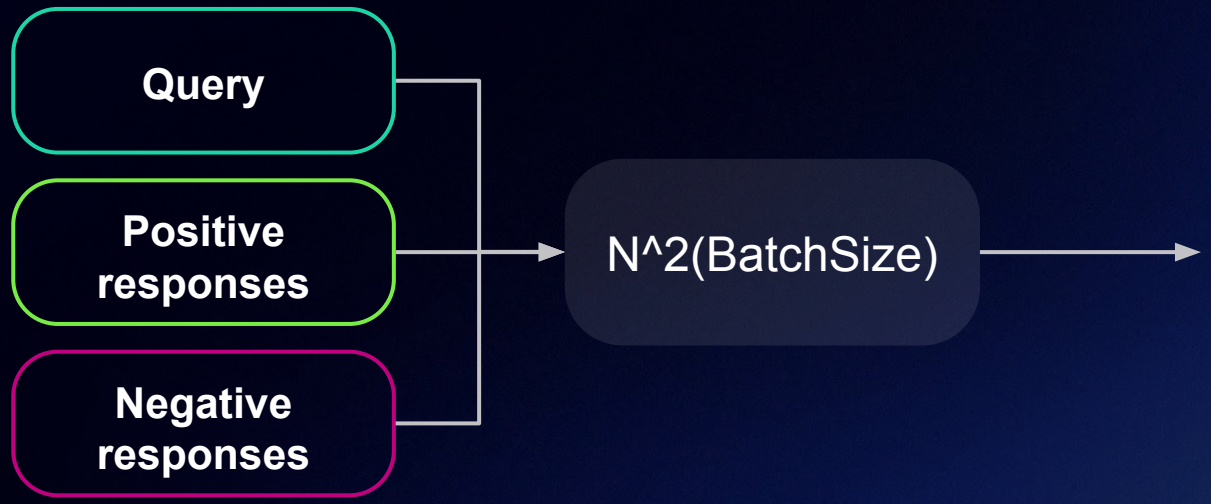
Семплинг обучения core NLU



Ускорение сходимости core NLU. FullBatching



Ускорение обучения core NLU. FullBatching



BatchSize ≤ 32,
но считаем в Loss N^2 комбинаций, используя всю информацию

+6% к скорости сходимости

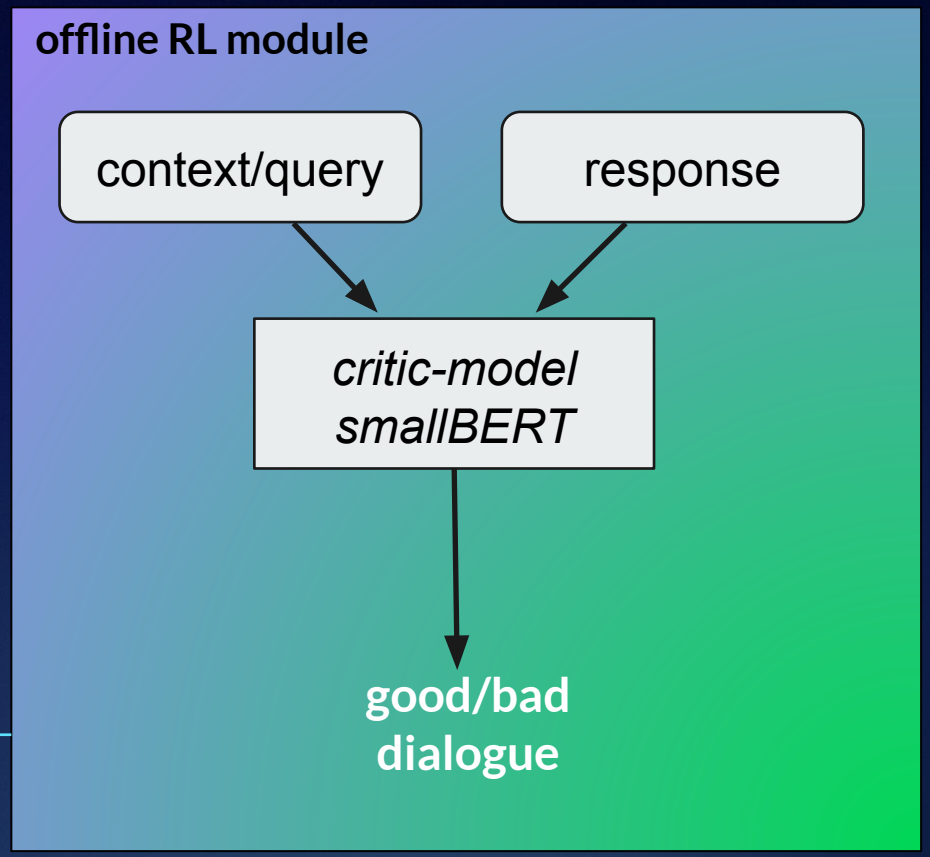
+4% к точности

Retrieval RL+Embedders with reward model

RL inspired adaptive "soft" margin loss for Embs:

Loss = max(0, NegSim - PosSim + r-score + margin)

relevance score



Retrieval RL+Embs+augmentex

— Мальчик ехал по полю
на автомобиле

Словарь синонимов русского языка

simple_aug



small-ELECTRA (tinyBERT, etc)

а) Мал_чЕк ехаль по полю
на _втомобиле

б) Юноша передвигался по полю
на тачке

offline RL module

context/query

response

critic-model smallBERT

good/bad dialogue

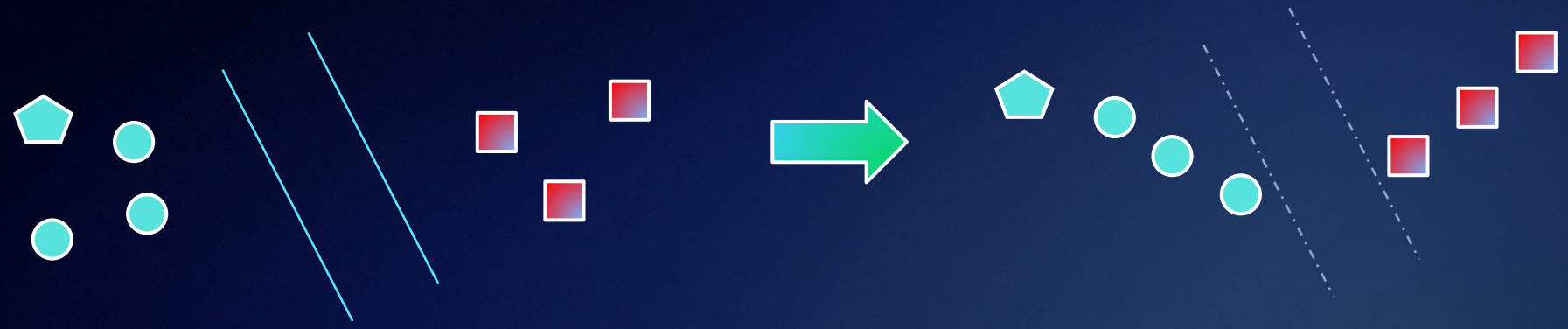
paraphrase Attack

Policy/Reward

Retrieval RL+ Embedders = улучшение эмбеддингов

Такая передача политик релевантности между вектором запроса/контекста и вектором ответа в функцию настройки их положения в пространстве позволяет:

1. + Внутри Pos и Neg пар более эффективно выстроить вектора относительно запроса
2. + Сделать “adaptive soft margin” между областями Pos и Neg векторов.

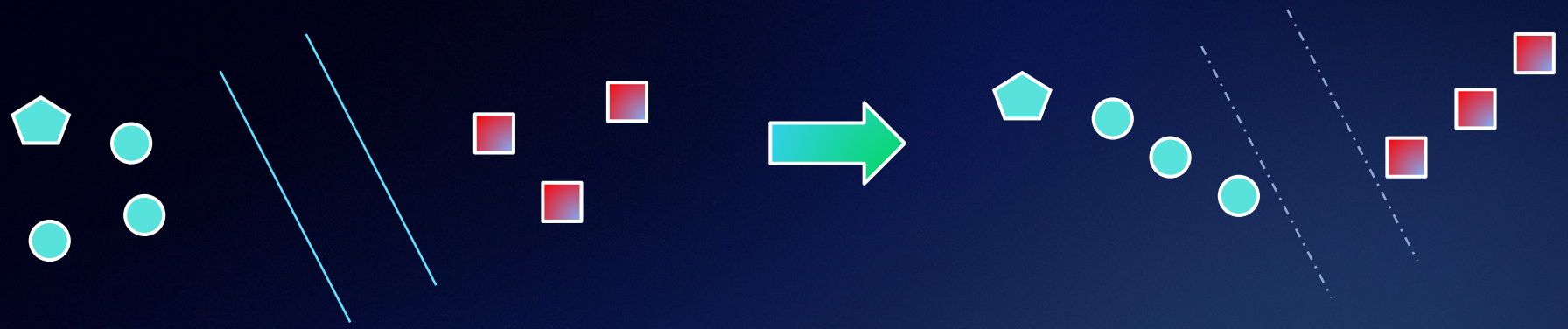


До

После

Retrieval RL+ Embedders = улучшение эмбеддингов

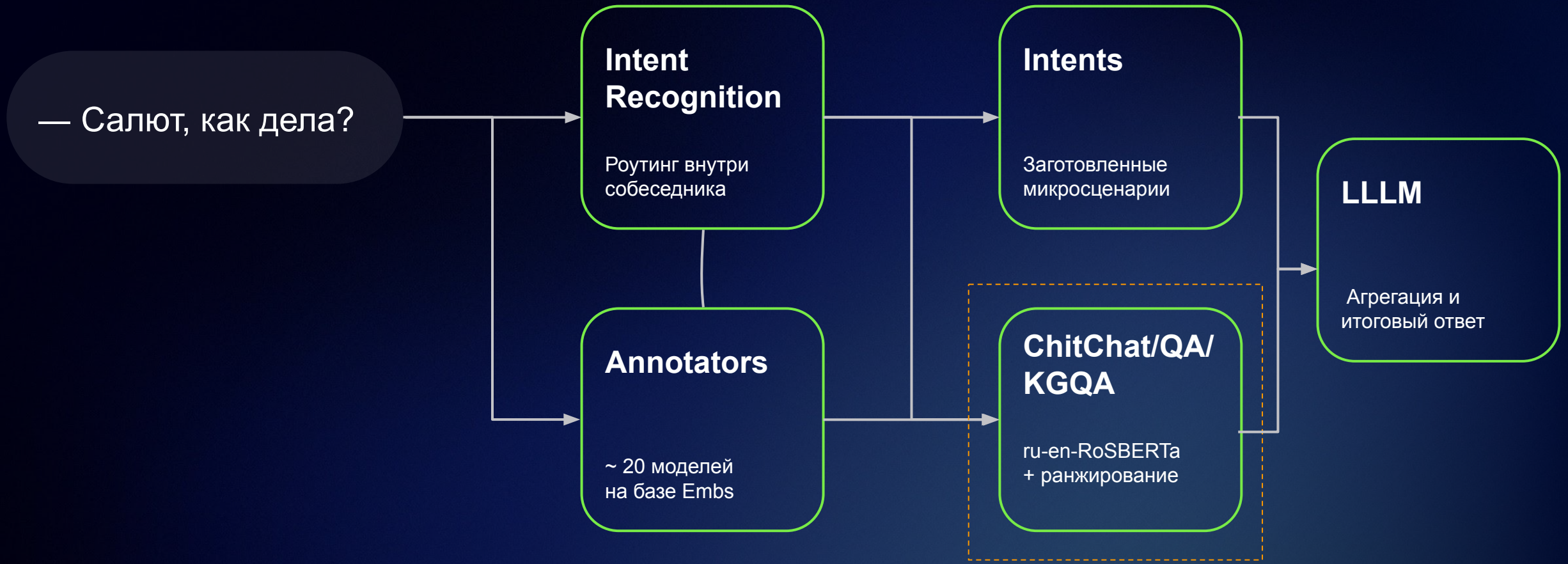
1. + Внутри Pos и Neg пар более эффективно выстроить вектора относительно запроса
2. + Сделать “adaptive soft margin” между областями Pos и Neg векторов.



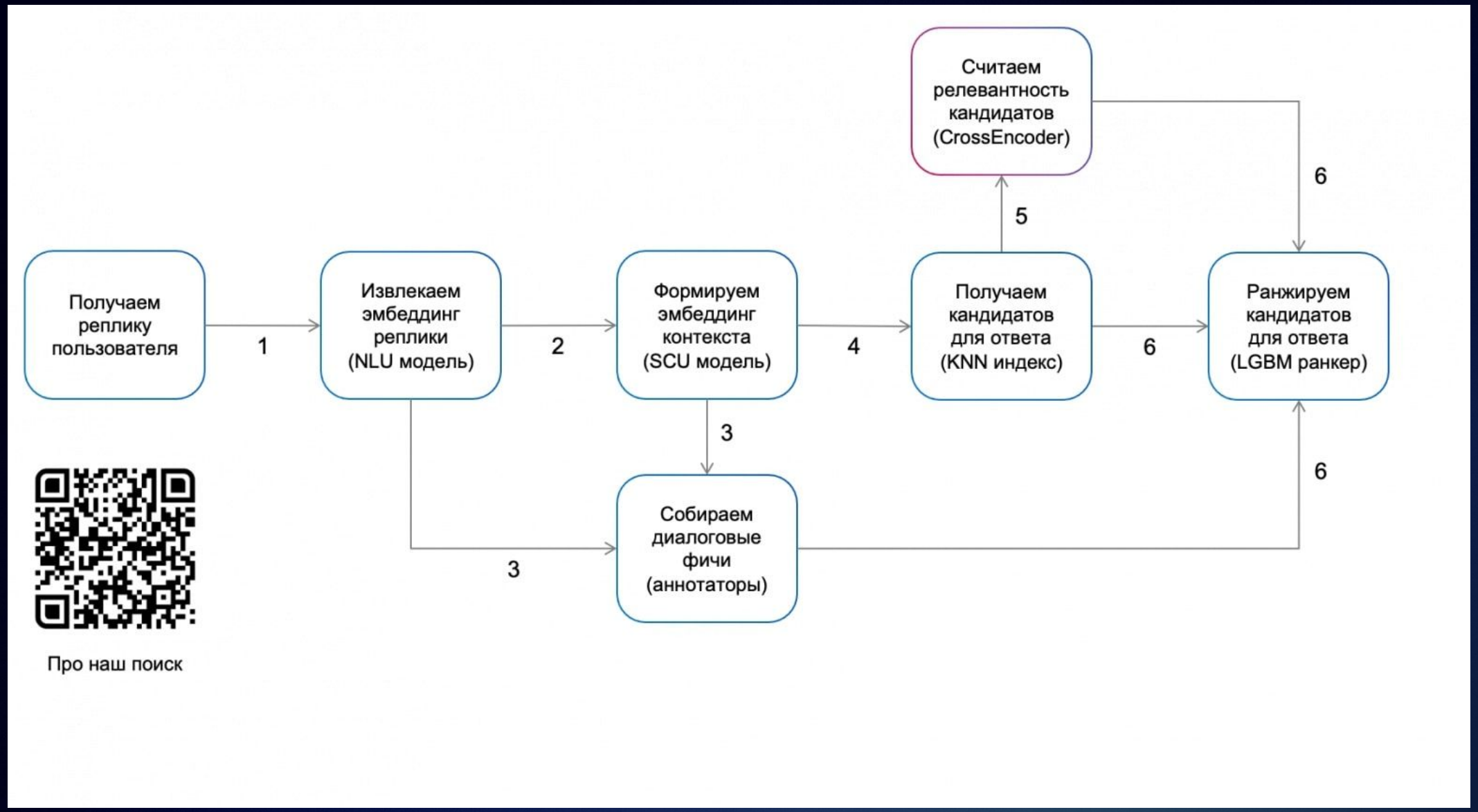
+15% к разнообразию топ1000
+4% к специчифности
+2% к релевантности

А что там с продом?

Общая схема работы e2e



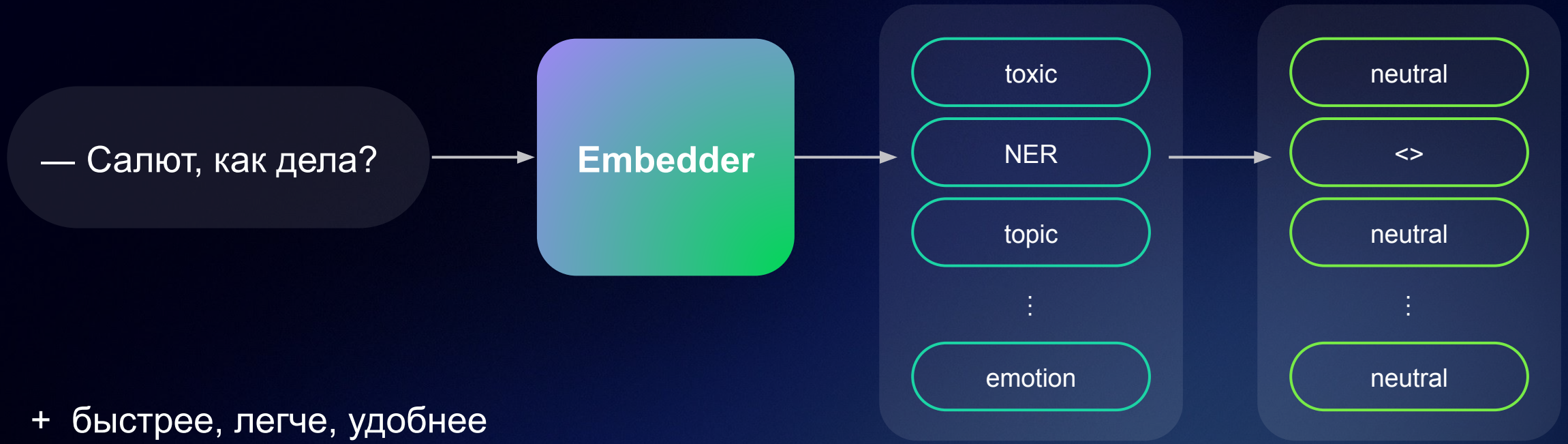
Общая схема работы поиска



Ускорение обучения. P-Learning и адаптеры



Векторизация и модели-адаптеры

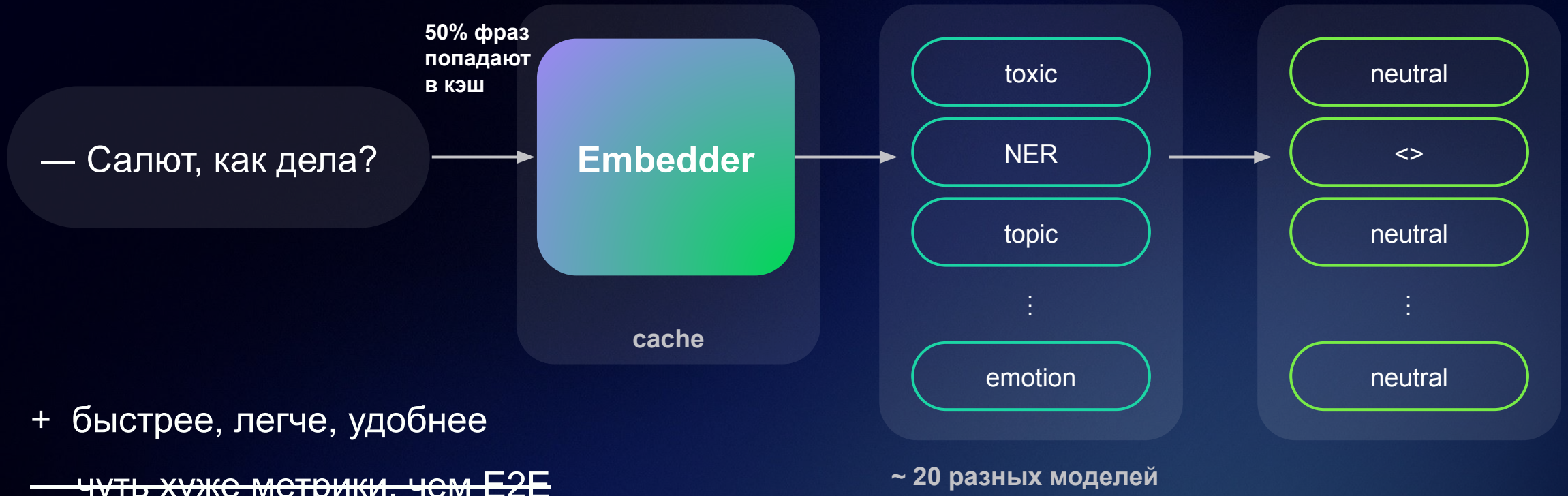


\+ быстрее, легче, удобнее

~~— чуть хуже метрики, чем E2E~~

— нужно следить за консистентностью

Векторизация и модели-адаптеры



\+ быстрее, легче, удобнее

~~— чуть хуже метрики, чем E2E~~

— нужно следить за консистентностью

FAISS для быстрого поиска

Facebook AI Similarity Search

```
import faiss                   # make faiss available
index = faiss.IndexFlatL2(d)   # build the index print(index.is_trained)
index.add(xb)                  # add vectors to the index
print(index.ntotal)

D, I = index.search(xq, k)     # actual search
print(I[:5])                   # neighbors of the 5 first queries
```

FAISS для быстрого поиска

300% быстрее поиск

IVF-индекс для ускорения

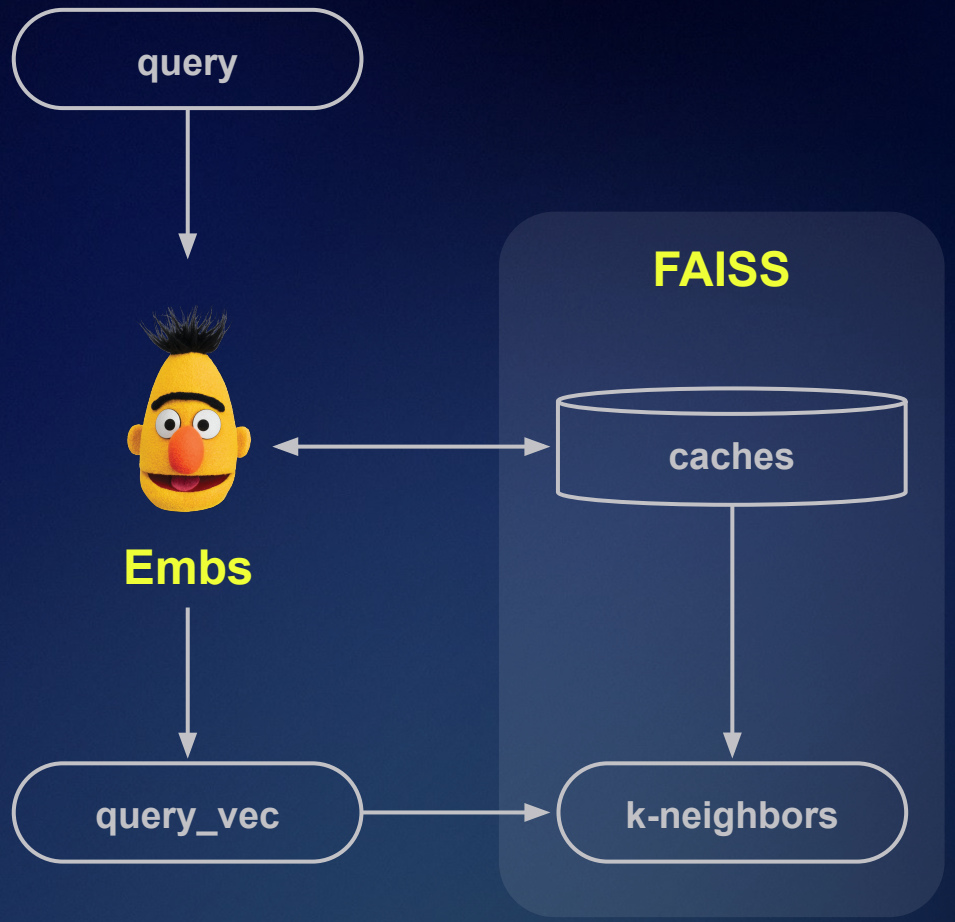
50% меньше памяти

Квантизация для уменьшения памяти

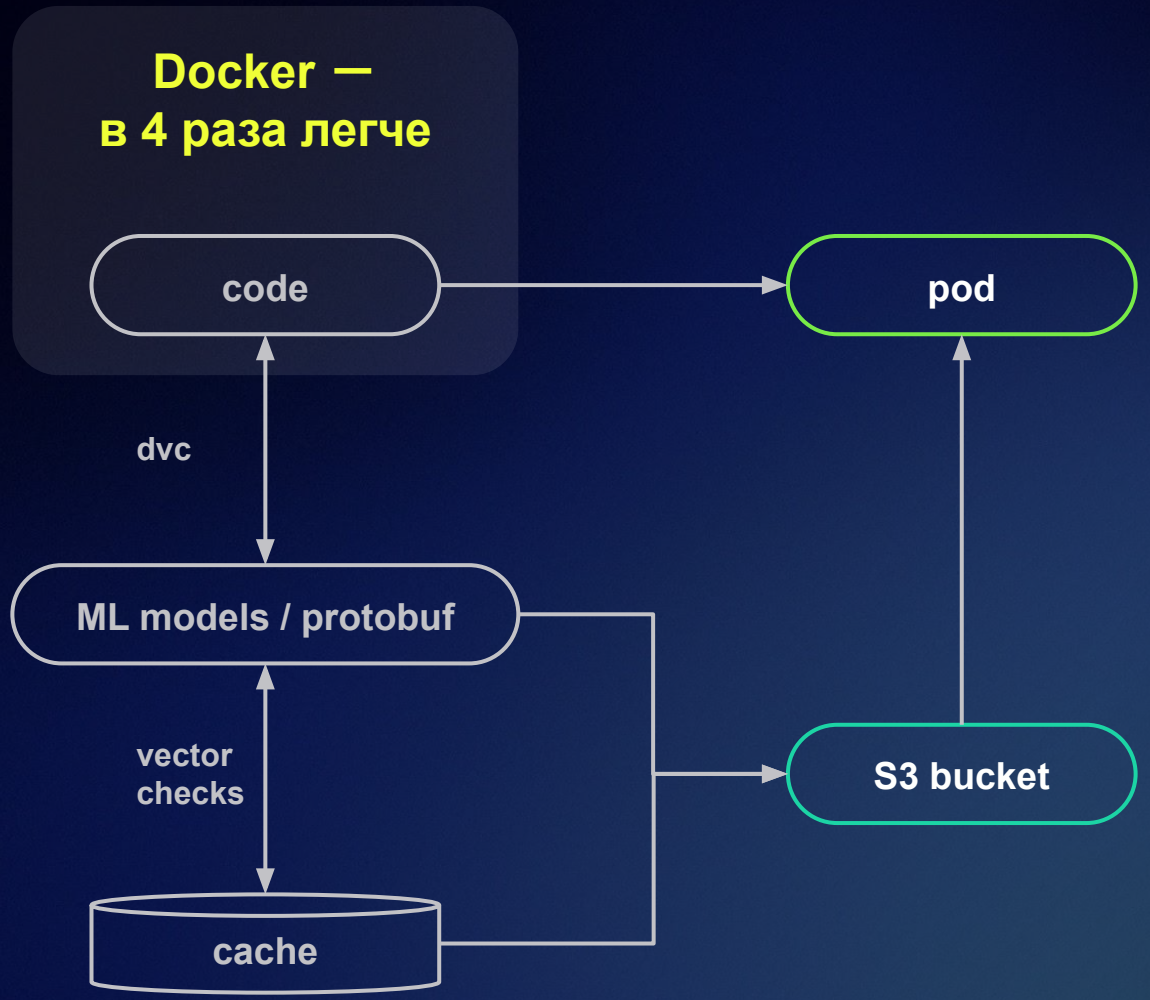
40% быстрее запуск

Шардирование для ускорения запуска

Сериализация, чтобы хранить и читать с диска



Консистентность



Ускорение и облегчение инференса

Скорость

- Latency & throughput
- Доставка изменений в прод

Память

- GPU
- Docker container

Принципы при внедрении нового ML-функционала

Metrics

— не хуже, чем было

Maintainability

— удобство поддержки

Reliability

— корректность, возможность откатиться

Adaptability

— быстрые итерации нововведений

Scalability

— масштабируемость

Итого

Скорость

Multitask

Adversarial Attacks

Distillation

Cold & Hot Cache

FAISS

CI/CD

Adapters

ONNX & TFfreeze formats

Память

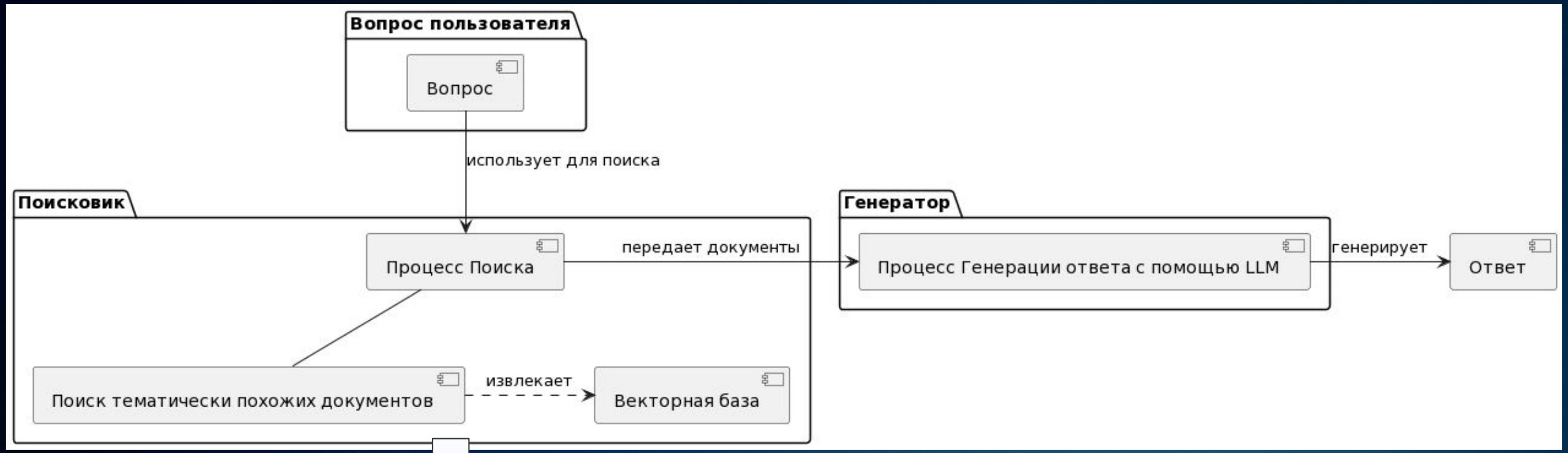
FullBatching

Docker

DimCompression

Хаки и хинты с RAG

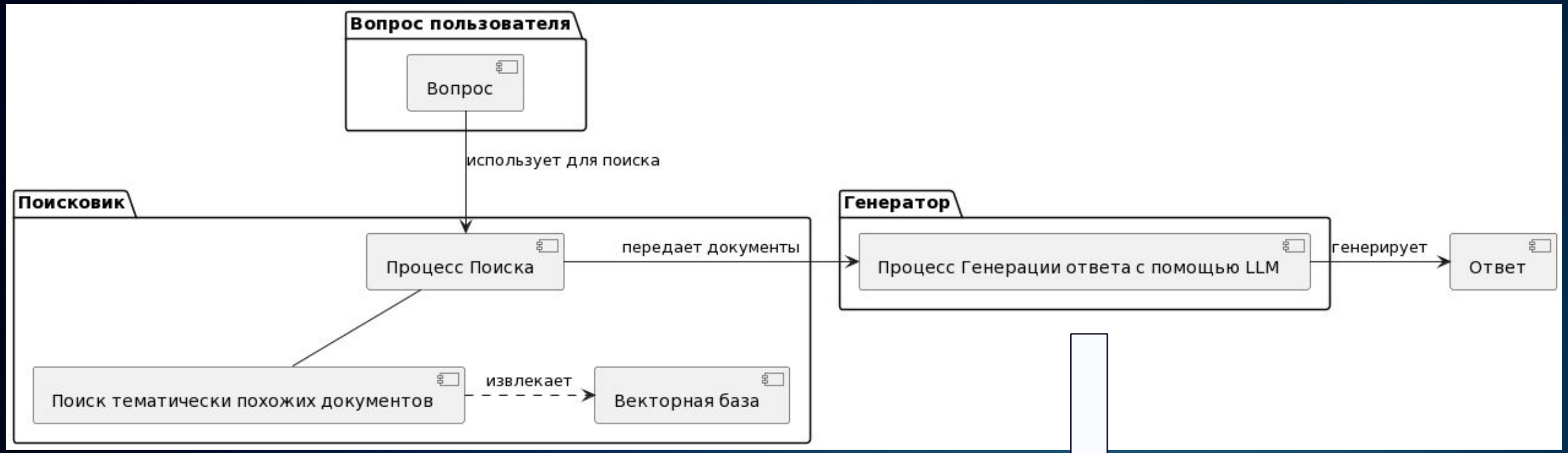
Как устроить хороший поиск - извлечение информации



<u>В качестве источников знаний могут быть:</u>

- Структурированные и не очень БД
- Текстовые документы: pdf,doc,txt, json и др.
- Картинки и видео/аудио файлы
- Web поиск: google, yandex, bing …

Как устроить хороший поиск - извлечение информации

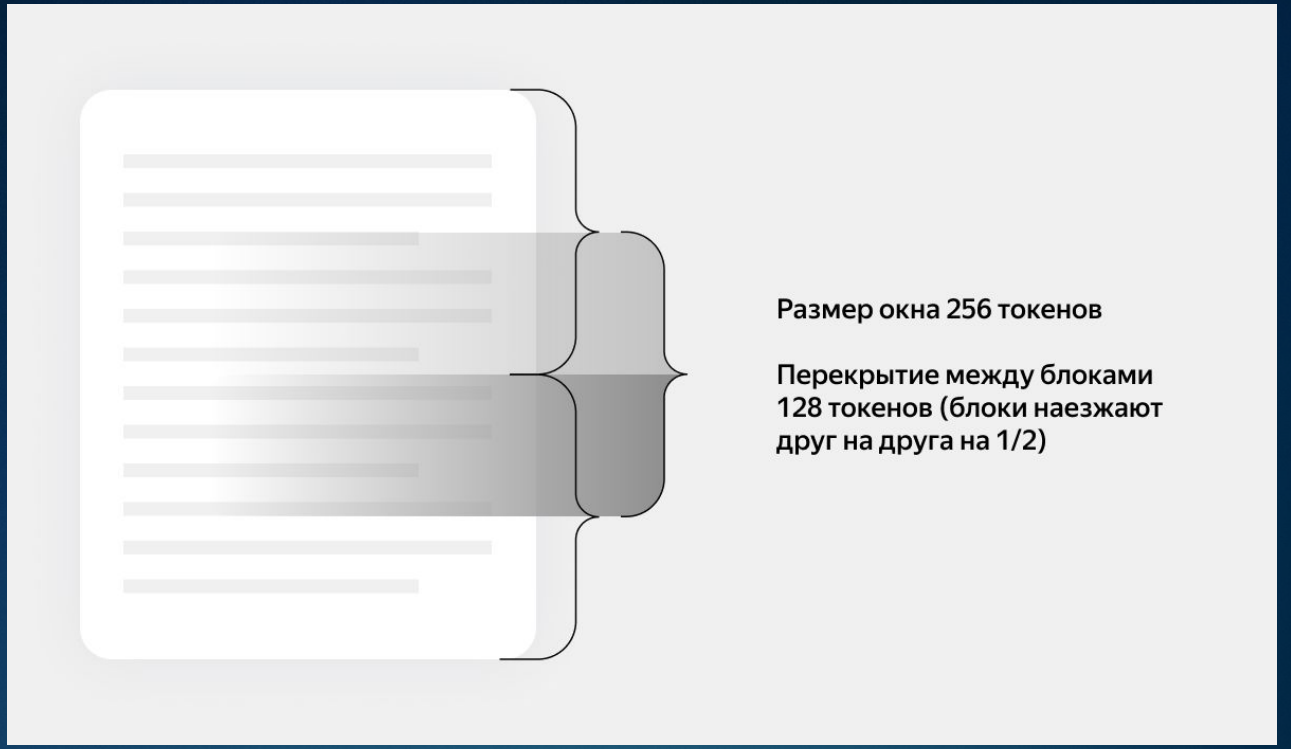


<u>В качестве моделей можно взять:</u>

- OpenAI или Google решения по Api
- Открытые решения LLM: DeepSeek, LLAMA X., Mistral, Qwen
- Пусть “сильных” - делать свое ;)

Как устроить хороший поиск - нарезка данных

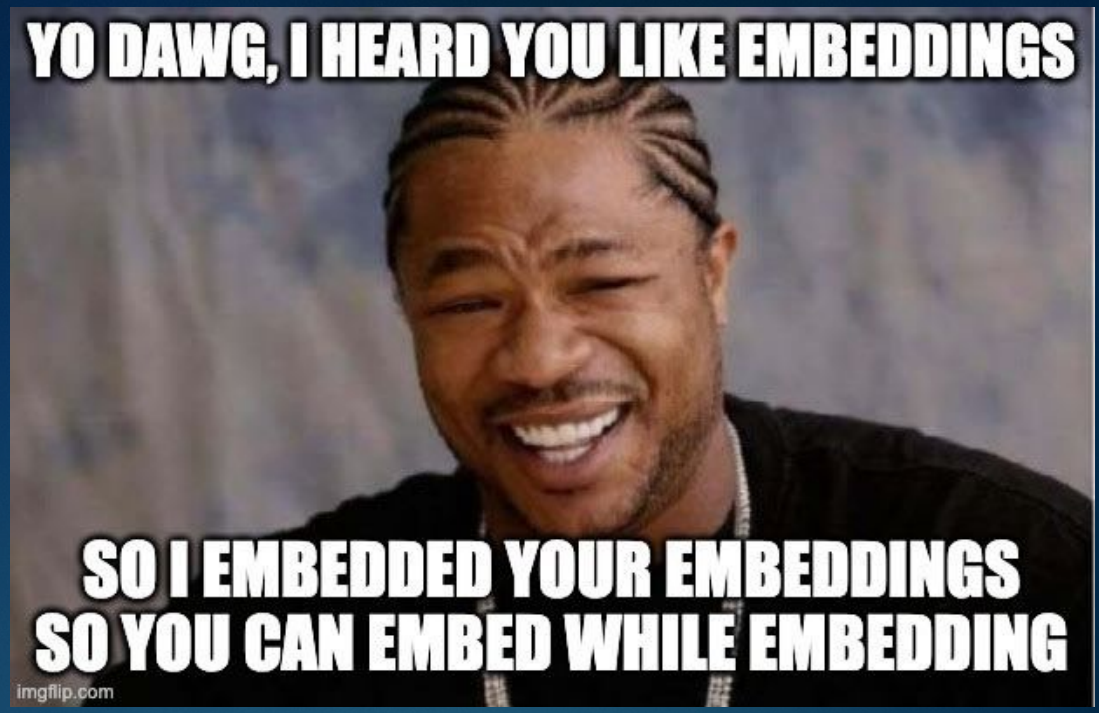
1. Найти источники хороших текстов
2. Возьмём текст документа.
3. Разобьём на блоки по 256 слов, при этом делаем так, чтобы они пересекались. Это нужно, чтобы не потерять важную информацию, которая могла оказаться на стыке двух блоков.
4. Тут этап оценки качества нарезки (+ еще одна модель или правила).
5. Выбираем самые релевантные блоки.
6. «Склеиваем» выделенные блоки в том же порядке, в котором они появились в исходном тексте и передаем в LLM.



*Пример с работы Yandex.

Как устроить хороший поиск - embedder

- Эмбеддер для поиска подсказок важен, с него растет качество. С ходу можно получить **прирост от 5-7%**.
- **Не** стоит **стесняться тюнить энкодер e2e без тюна LLM** (следим за конечной e2e метрикой).
- Так же **тюнить соло без e2e тоже**, как шаг 2 (после шаг1 бейзлайна взять лучший энкодер по MTEB), а e2e тюн с LLM уже последняя мера.
- **Реранкер** может **улучшить** качество **топ-К** кандидатов.



Где искать хороший embedder? На ruMTEB!

MTEB(rus, v1)

A Russian version of the Massive Text Embedding Benchmark with a number of novel Russian tasks in all task categories of the original MTEB.

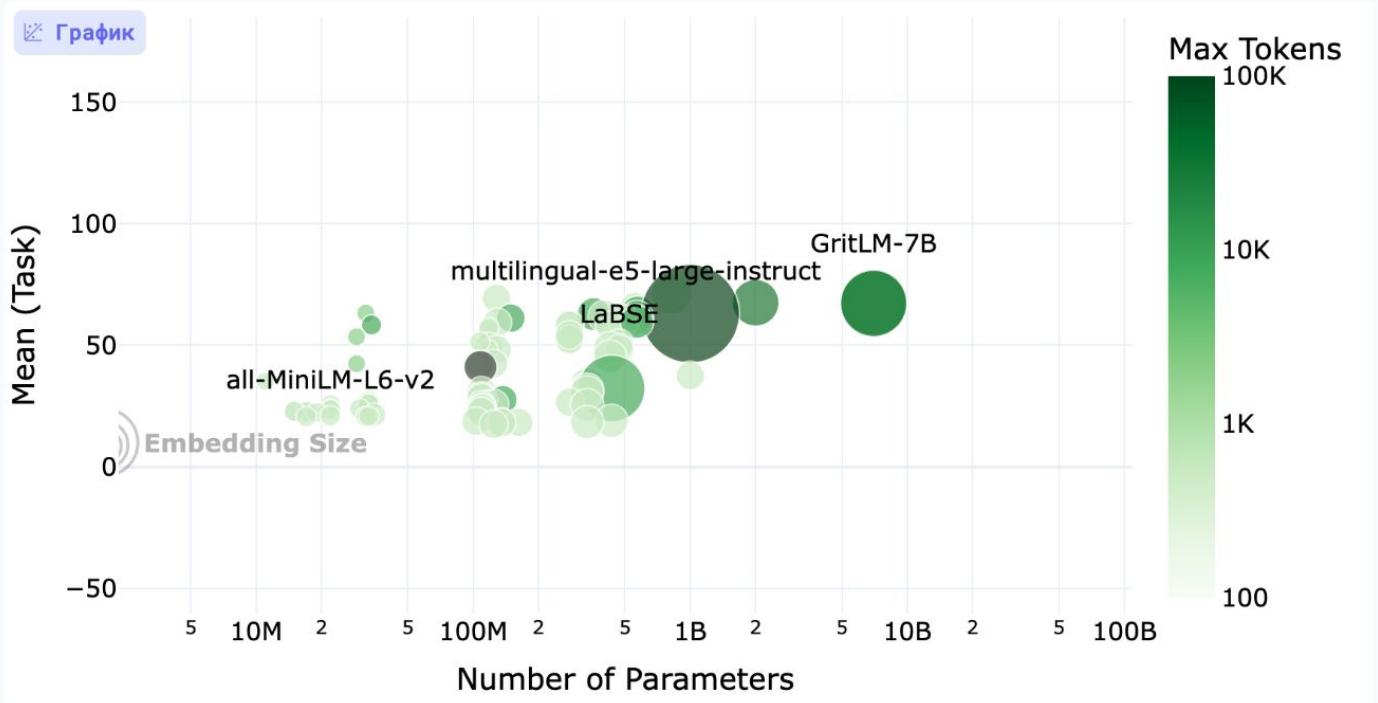
- **Number of languages**: 1
- **Number of tasks**: 23
- **Number of task types**: 7
- **Number of domains**: 9

Click for More Info

Cite this benchmark: ◀

Share this benchmark: ◀

Performance per Model Size | Performance per Task Type (Radar Chart)



We only display models that have been run on all tasks in the benchmark

Customize this Benchmark ◀

Advanced Model Filters ◀

Где искать хороший embedder? На ruMTEB!

Summary | Performance per task | Task information

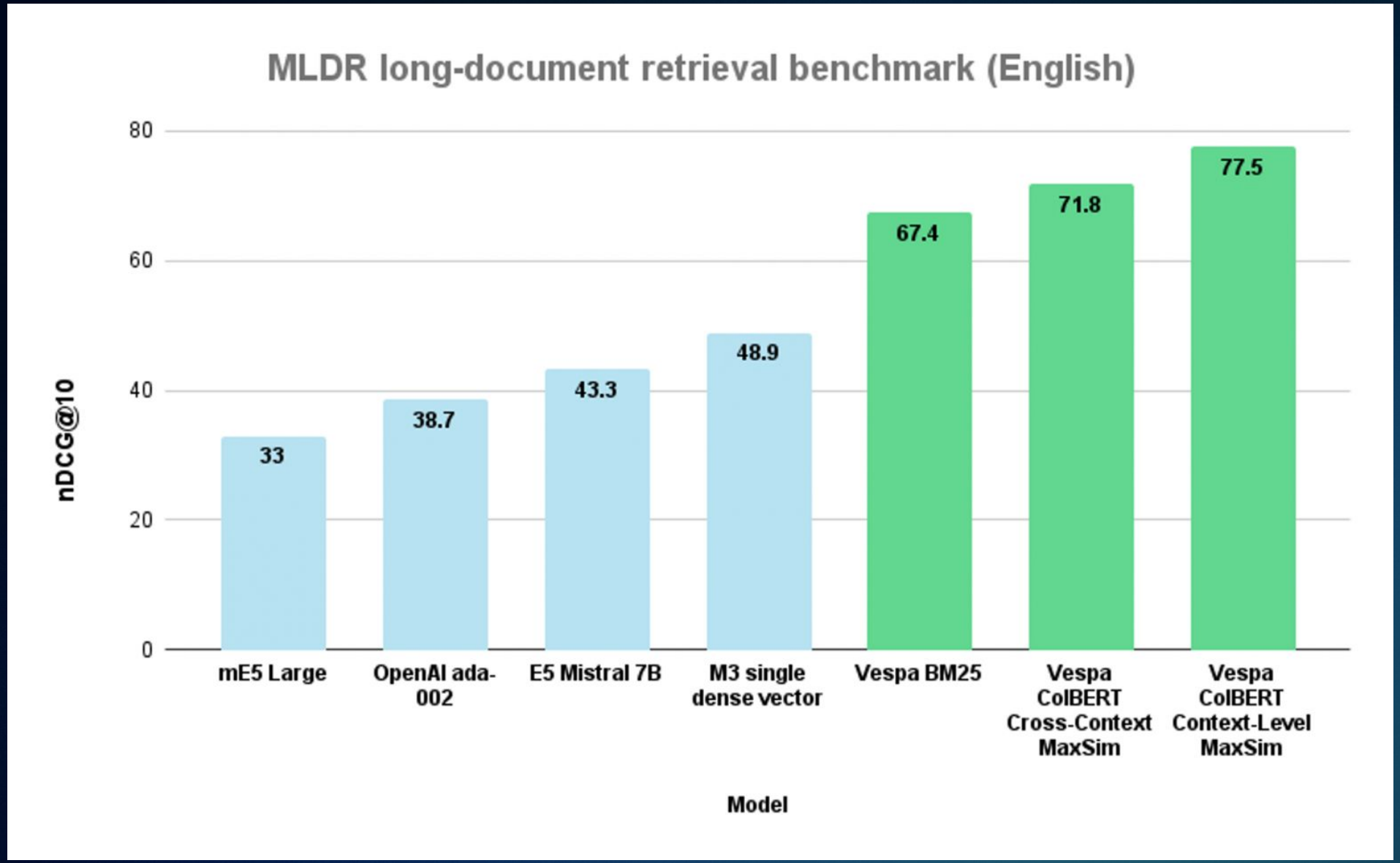
Filter...

Rank (Bo…	Model	Zero-shot	Memory U…	Number of P…	Embedding D…	Max Tokens	Mean (T…	Mean (TaskT…	Classification	Clust
1	FRIDA	30%	3141	823M	1536	512	**71.06**	**68.98**	73.71	**67.49**
2	BERTA	30%	489	128M	768	512	69.34	67.86	71.22	64.78
3	Linq-Embed-Mistral	91%	13563	7B	4096	32768	67.46	64.62	71.16	62.92
4	GritLM-7B	91%	13813	7B	4096	4096	67.67	65.08	69.92	64.30
5	e5-mistral-7b-instruct	91%	13563	7B	4096	32768	67.16	64.99	69.07	64.24
6	Giga-Embeddings-instruct	⚠ NA	9649	2B	2048	32768	67.49	64.02	**73.75**	61.79
7	multilingual-e5-large-instruct	91%	1068	560M	1024	514	65.00	63.36	66.28	63.13
8	SFR-Embedding-Mistral	86%	13563	7B	4096	32768			69.81	64.92
9	gte-Qwen2-7B-instruct	⚠ NA	29040	7B	3584	32768			72.97	66.37
10	jina-embeddings-v3	100%	1092	572M	1024	8194	63.45	60.93	65.24	60.90
11	SFR-Embedding-2_R	86%	13563	7B	4096	32768			70.20	64.87

Download Table

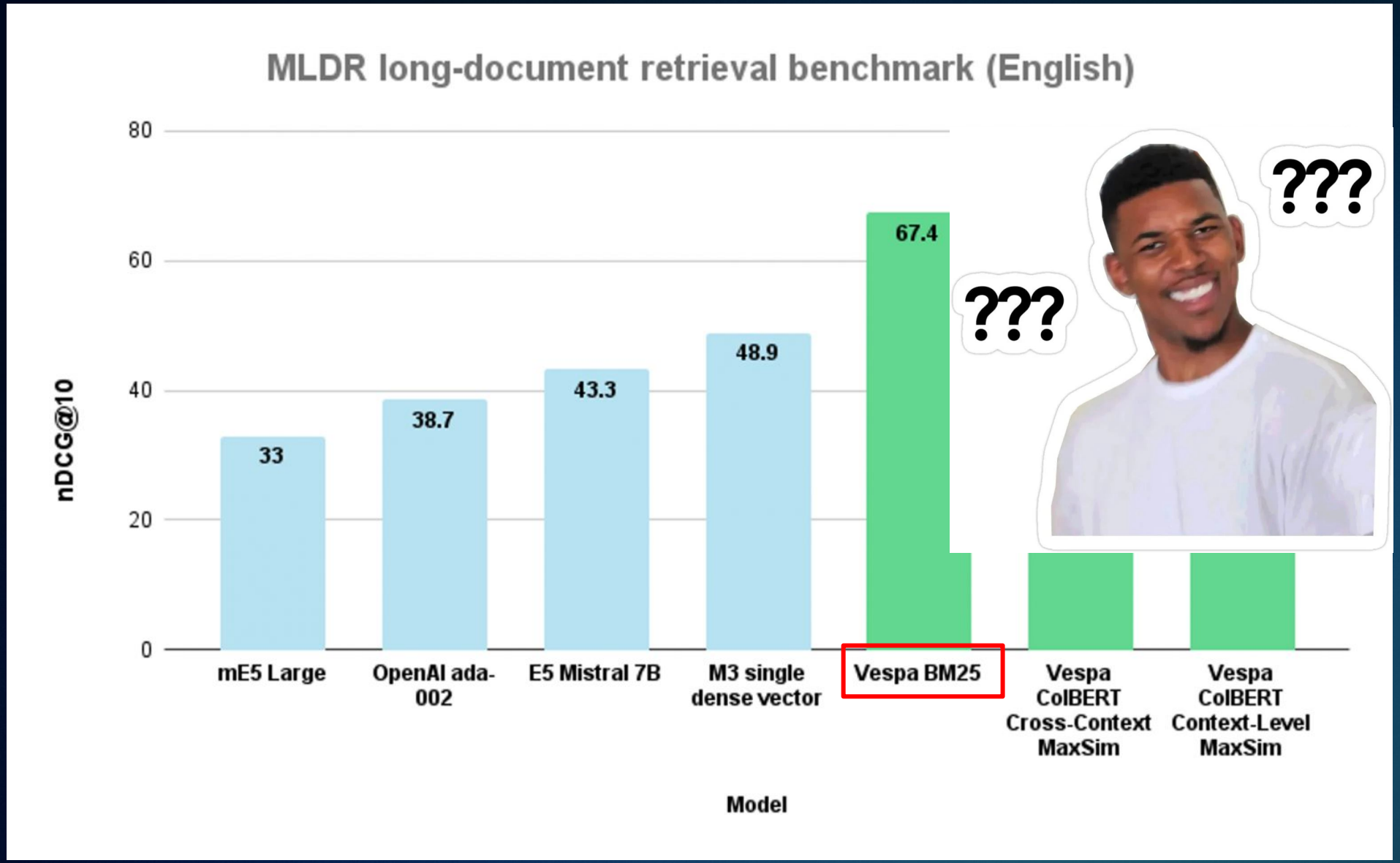
Frequently Asked Questions ◀

Как устроить хороший поиск - BM25/TF-IDF



https://blog.vespa.ai/announcing-long-context-colbert-in-vespa/

Как устроить хороший поиск - BM25/TF-IDF



https://blog.vespa.ai/announcing-long-context-colbert-in-vespa/

Как устроить хороший поиск - MMR или “прореживание”

MMR* поверх выдачи эмбеддера **повышает разнообразие** итоговых **подсказок** и даёт лучший импакт на конечную генерацию.

Идея MMR в том, что нам нужно получить **не топК ближайших** парафраз/формулировок **подсказок**, а **топК разнообразных релевантных подсказок**, не являющихся парафразами друг-друга. Это и помогает сделать MMR метрика.



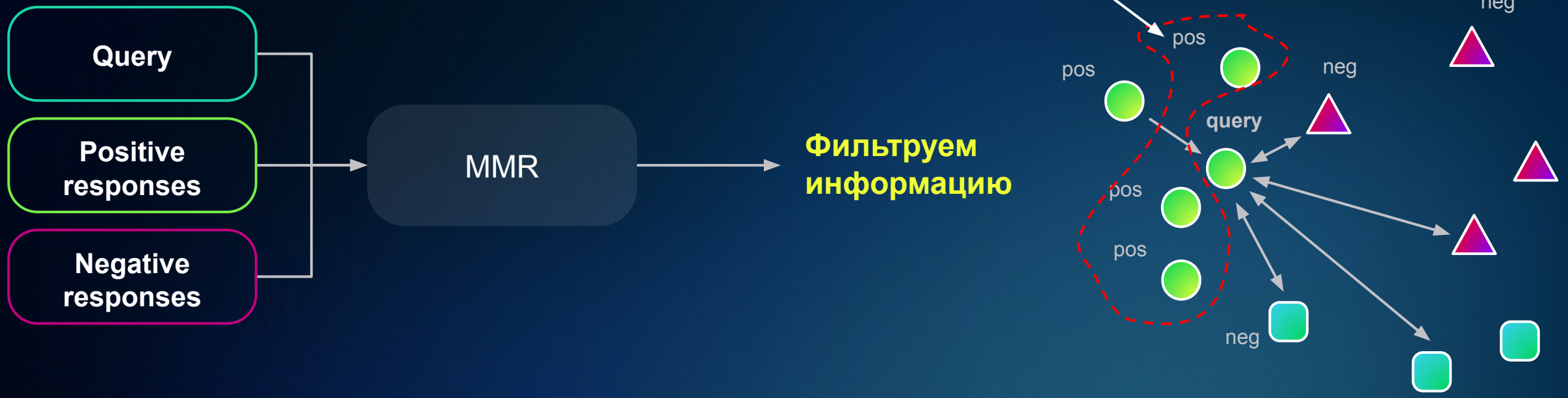
* источник https://aclanthology.org/X98-1025.pdf

Как устроить хороший поиск - MMR или “прореживание”

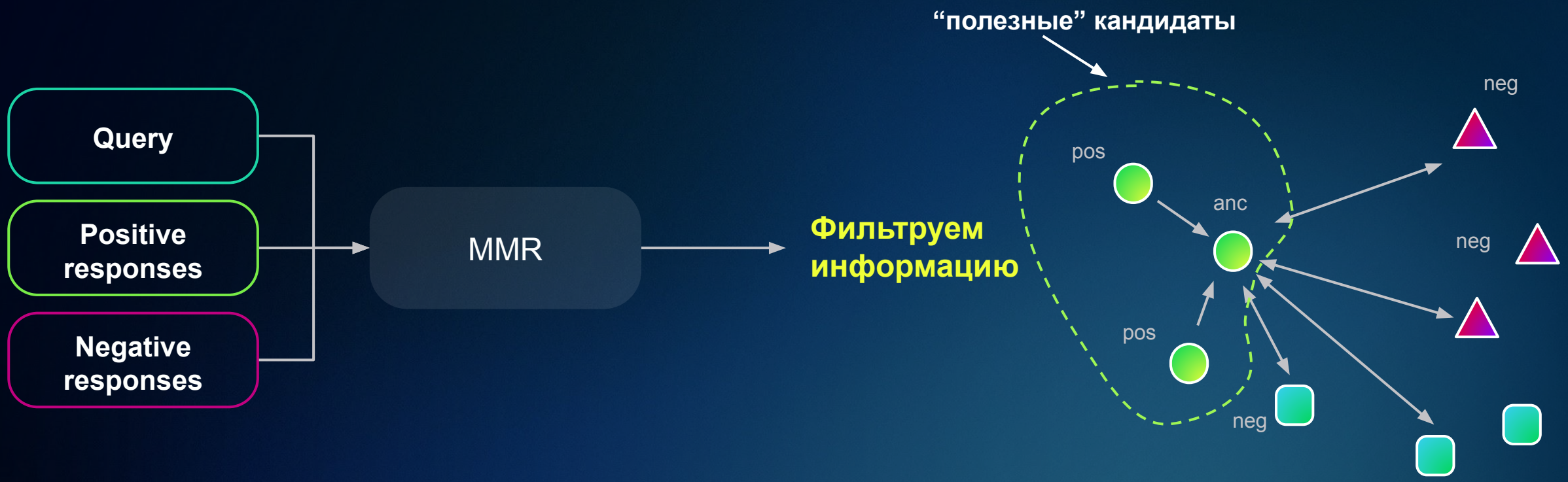


* источник https://aclanthology.org/X98-1025.pdf

Как устроить хороший поиск - MMR или “прореживание”



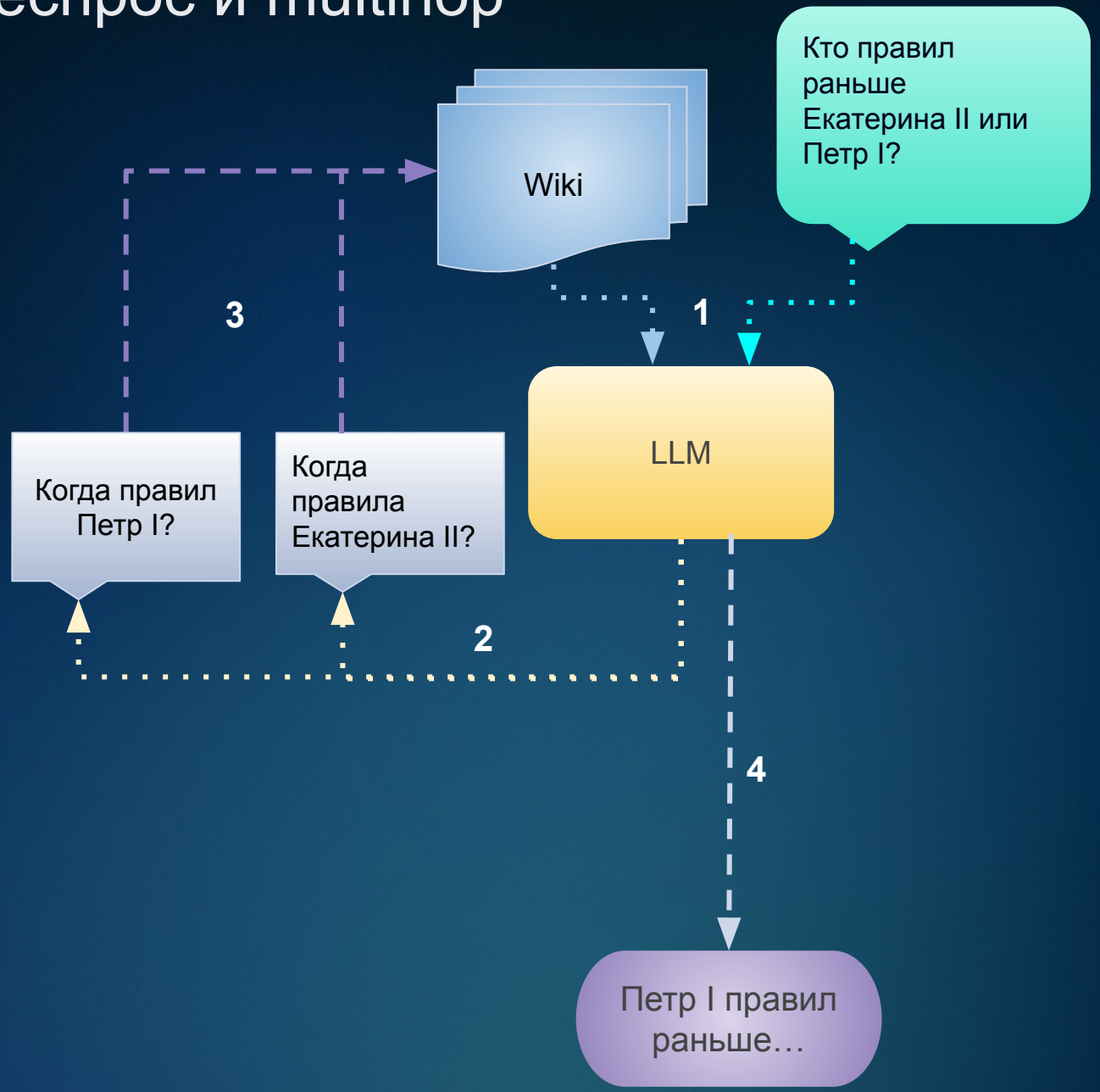
Как устроить хороший поиск - MMR или “прореживание”



Как еще улучшить RAG — переспрос и multihop

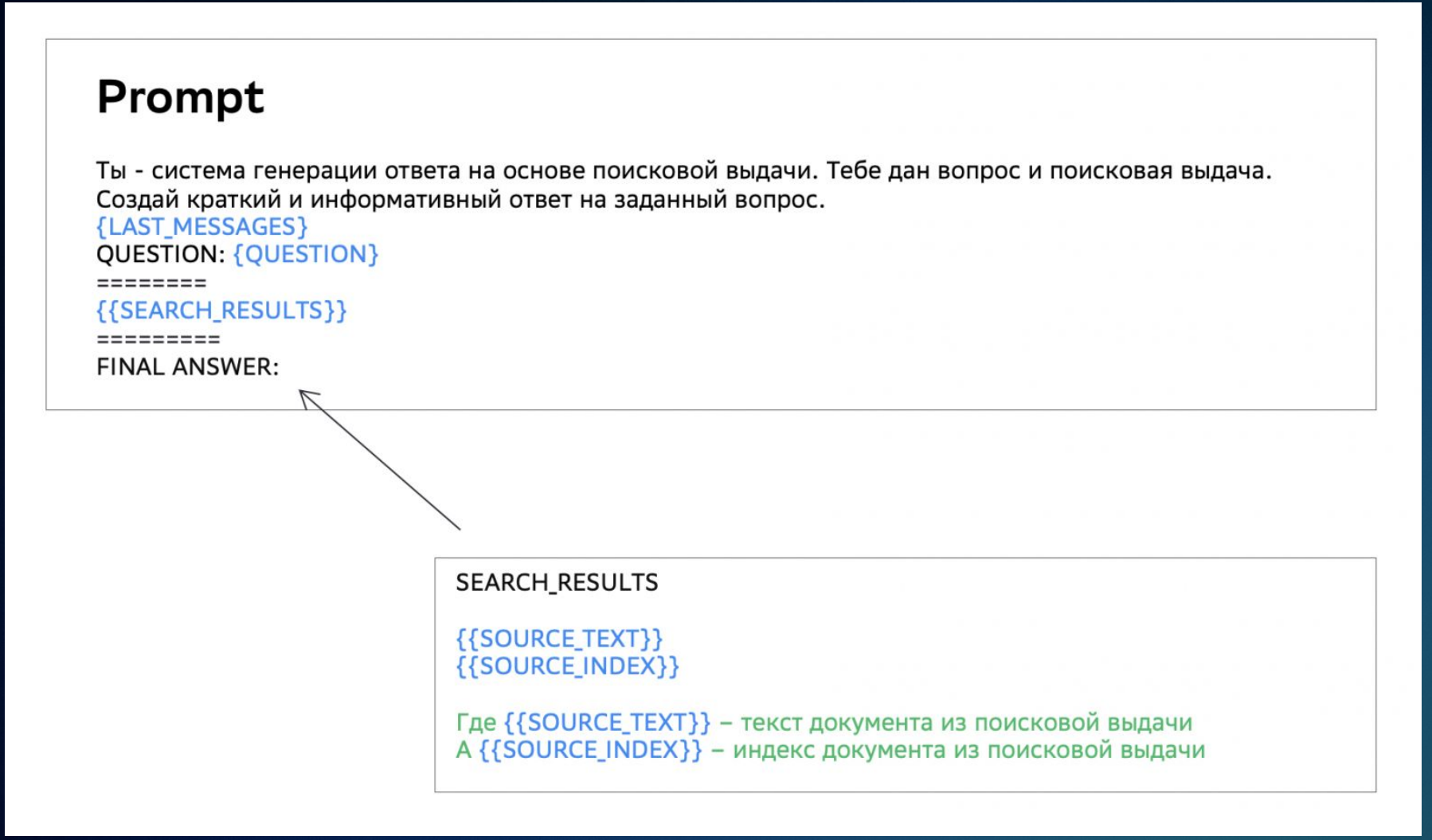
Иногда, **LLM сложно ответить на составной вопрос**, требующий уточнений. Но **мы можем инициировать разбиение** составного **вопроса** на несколько уточняющих, так и появился **multihop RAG**.

К примеру, чтобы узнать **отношение между событиями в датах**, нужно уточнить, когда происходило каждое событие и предоставить данную информацию LLM для вывода.



Как еще улучшить RAG — prompting

Промтинг всё ещё также в купе с пп. 1-2 даёт импакт к итоговой генерации. Тут можно **получить +2% минимум** к качеству ответа.



Про метрики RAG

Как оценивать RAG?

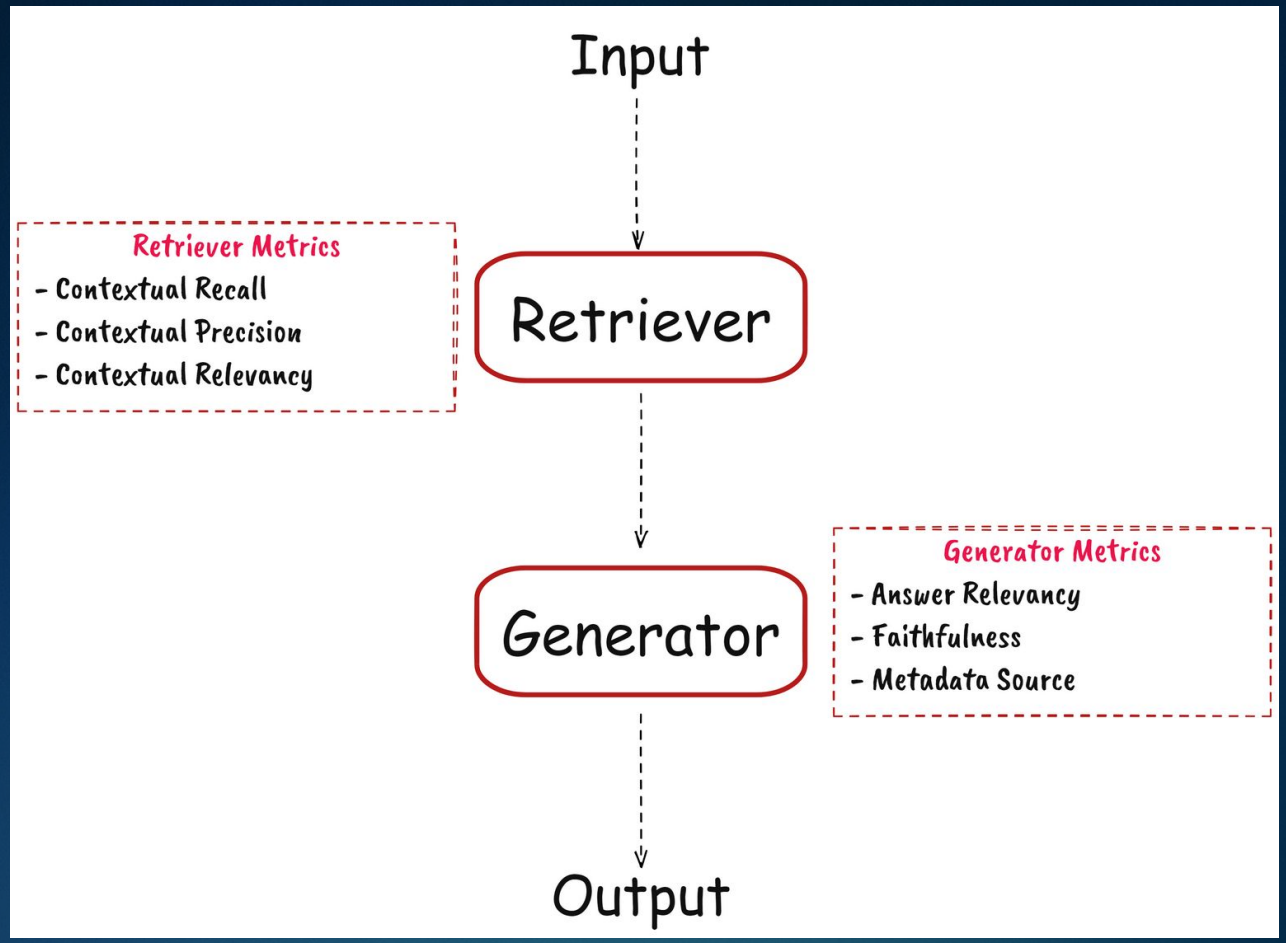
Всегда есть типы ошибок, которые рассматриваются как:

1. **Ошибка поиска @k**, когда в топ-k выдачи нет полезных подсказок.
2. **Ошибка ризонинга**,когда LLM получила подсказки, но не ответила.

Ошибку ризонинга разделяют:

1. На ошибку, когда в топе была подсказка и модель не ответила,
2. Когда подсказки не было, и LLM не ответила (т.е. не сработала из весов)*.

**Самый интересный кейс, когда модель сама принимает решение ответить из весов, несмотря на плохие подсказки.*



Как оценивать RAG? Ошибка поиска.

Всегда есть типы ошибок, которые рассматриваются как:

1. **Ошибка поиска** @k, когда в топ-k выдачи есть/нет полезных подсказок:

$$precision@k = \frac{|R_u(k) \cap L_u|}{|R_u(k)|} = \frac{correctly\ recommended\ items}{total\ recommended\ items},$$

Как оценивать RAG? Ошибка поиска.

Всегда есть типы ошибок, которые рассматриваются как:

2. **Дисконтированная ошибка поиска @k**, когда в топ-k выдачи нужно правильное положение подсказок:

Этот недостаток нивелирует метрика ранжирования average precision at K (ap@K), которая равна сумме p@k по индексам k от 1 до K только для релевантных элементов, деленному на K

$$ap@K = \frac{1}{K}\sum_{k=1}^{K} r^{true}(\pi^{-1}(k)) \cdot p@k.$$

Пример:

Так, если из трех элементов выдачи релевантным оказался только находящийся на последнем месте, то *ap@3 = ⅓*(0/1+0/2+1/3*, если угадали лишь тот, что был на первом месте, то a*p@3 = ⅓*(1/1+0/2+0/3)*, а если угаданы были все, то *ap@3 = ⅓*(1/1+2/2+3/3)*.

Как оценивать RAG? Ошибка поиска.

Всегда есть типы ошибок, которые рассматриваются как:

1. Дисконтированная ошибка поиска @k, когда в топ-k выдачи нужно правильное положение подсказок:

   Этот недостаток нивелирует метрика ранжирования average precision at K (ap@K), которая равна сумме p@k по индексам k от 1 до K только для релевантных элементов, деленному на K

$$ap@K = \frac{1}{K}\sum_{k=1}^{K} r^{true}(\pi^{-1}(k)) \cdot p@k.$$



$$MAP@k = \frac{1}{n_{users}} AP@k$$

Как оценивать LLM в RAG? Разметка экспертами.

Базовый популярный подход к оценке **ошибки рассуждений LLM** или **e2e качества** RAG - это **ручная разметка**. Для оценки данным методом необходимо сформировать **дизайн-документ с правилами разметки**, а также иметь **golden-set.**

0. Технические метрики качества контента

0.1 Чат-ответы (LLM)

Метрика	Определение	Шкала 1–5
Relevance	Насколько ответ совпадает с темой и вопросами пользователя.	1 – полностью нерелевантно ⋯ 5 – идеально
Completeness	Покрытие всех ключевых аспектов запроса.	1 – критические пробелы ⋯ 5 – полный охват
User Satisfaction	Оценка пользователя диалога в целом.	1 – крайне неудовлетворён ⋯ 5 – полностью доволен

0.2 Поисковая выдача

Метрика	Определение	Шкала 1–5
Relevance	Соответствие результатов введённым ключевым словам и намерению.	1 – нерелевантно ⋯ 5 – идеально
User Satisfaction	Оценка пользователем полезности выдачи.	1 – бесполезно ⋯ 5 – полностью устроило

Метрики для краткого саммари отзывов

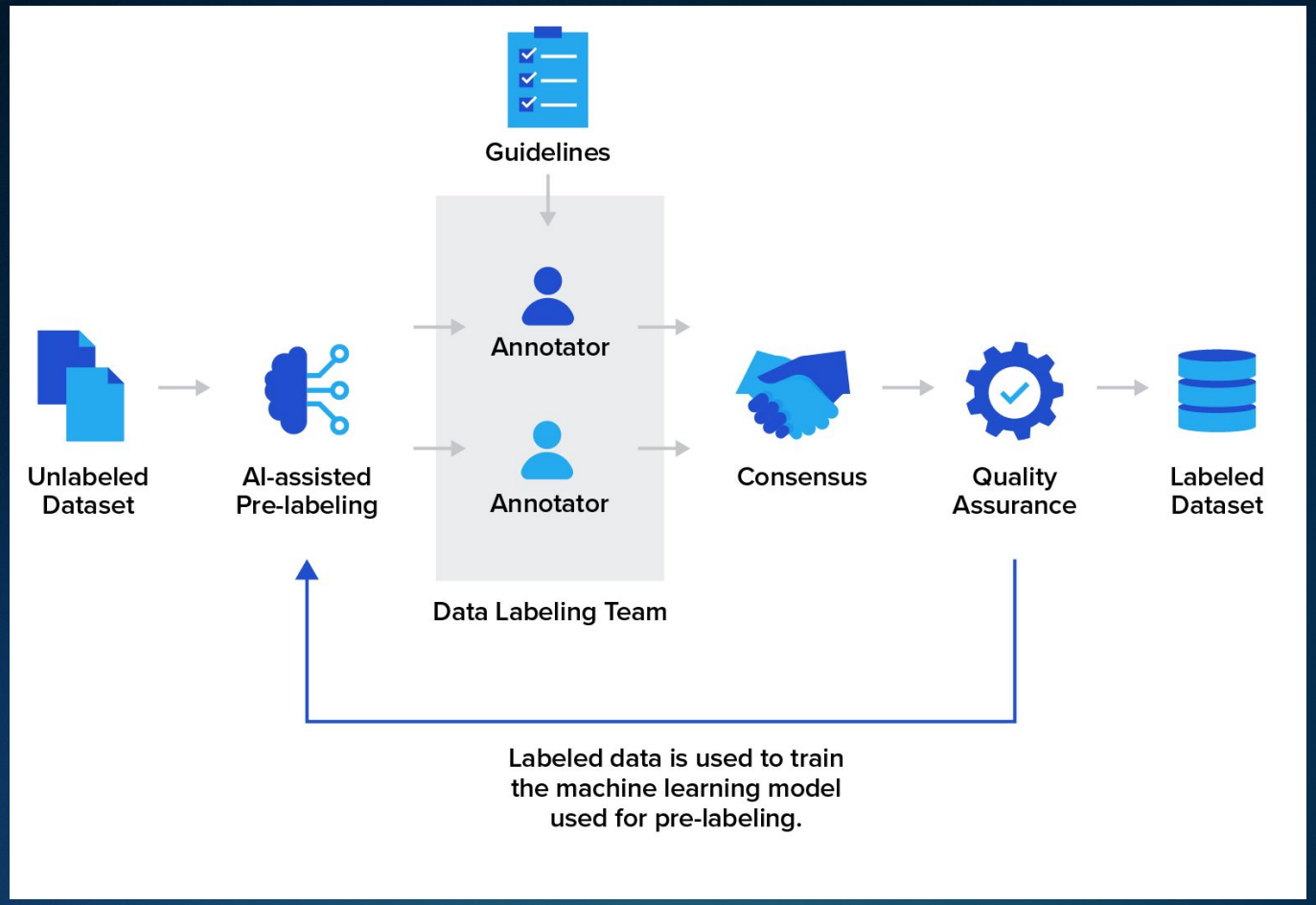
1. **Релевантность саммари**
   - Насколько информация в саммари соответствует ключевым темам и особенностям, поднимаемым в отзывах.
2. **Полнота саммари**
   - Насколько саммари охватывает весь спектр мнений, включая разнообразие точек зрения, положительные и отрицательные аспекты.
   - Оценивается, не упускаются ли важные аспекты, упомянутые в отзывах.
3. **Информативность саммари**
   - Оценивается, насколько саммари концентрирует важную информацию из исходных отзывов.
   - Проверяется доля уникальных ключевых аспектов (положительных, отрицательных, нейтральных), отраженных в саммари, относительно всех ключевых аспектов в исходных данных
4. **Читабельность саммари**
   - Насколько текст саммари легко читается, логично структурирован и грамматически правильный.
   - Проверяется, насколько итоговый текст удобен для восприятия, особенно для быстрой оценки.

Примеры инструкций разметки

Как оценивать LLM в RAG? Разметка экспертами.

Как выглядит “честный” алгоритм разметки:

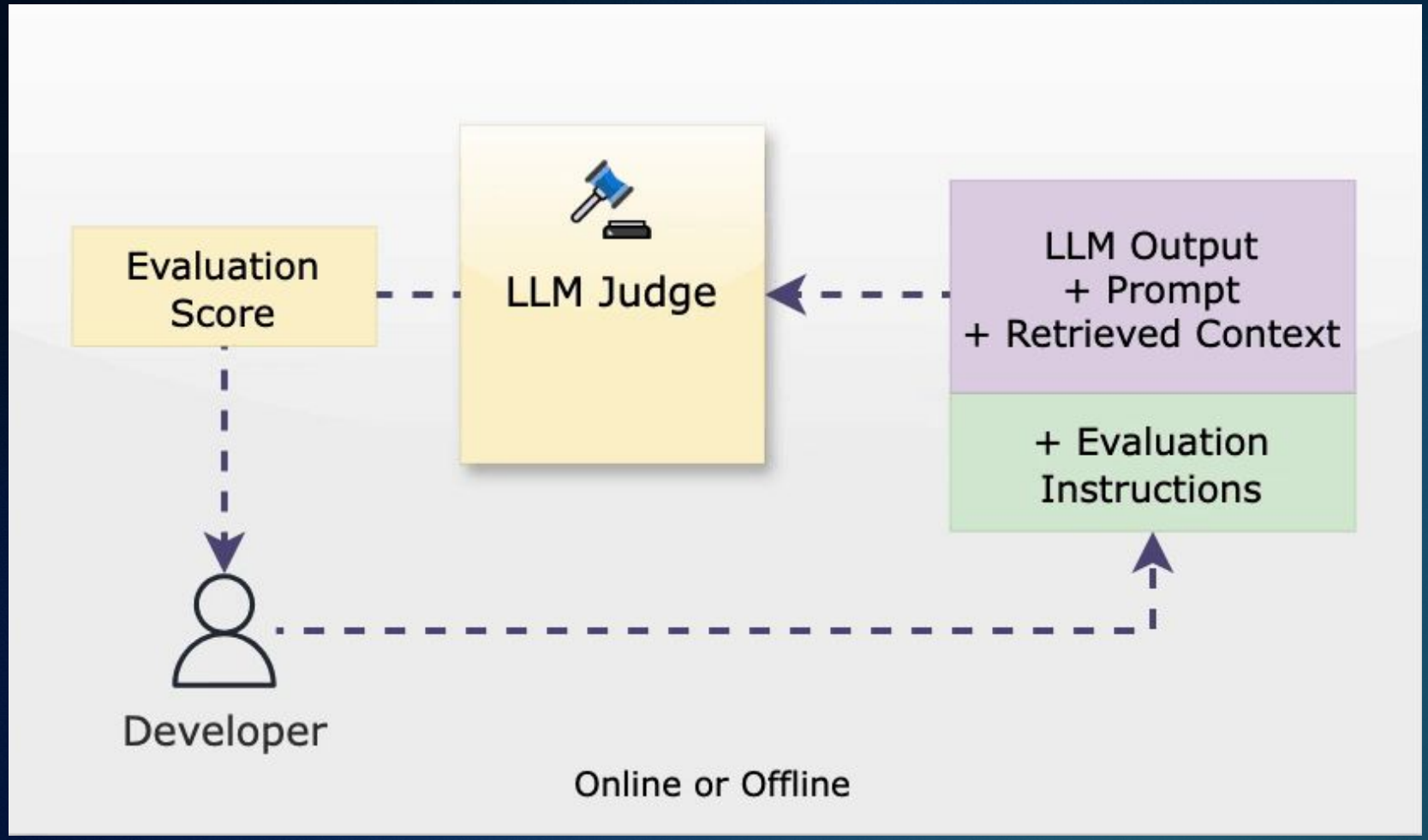
1. Выбирается K разметчиков. Обычно 3-5 человек или более, но всегда нечетное число.
2. Разметчики подбираются независимо, чтобы не было предвзятости и влияния друг на друга.
3. Датасет для разметки делят на пересекающиеся куски “внахлест” и раздают K экспертам.
4. Эксперты следуя инструкции размечают свой кусочек.
5. Т.к. данные были внахлест каждый кусочек имеет K независимых оценок от разметчиков.
6. Итоговую оценку выбирают или усреднением или по большинству голосов.



Типовой пайплайн оценки

Как оценивать LLM в RAG? LLM as judge.

Самый популярный подход к оценке **ошибки ризонинга** или **e2e качества** RAG - это **LLM as judge**. Для оценки данным методом необходимо сформировать “оценочный” промпт, а также иметь **golden-set.** Он также требует инструкции, которая ляжет в промпт, зато без дорогой разметки людьми.



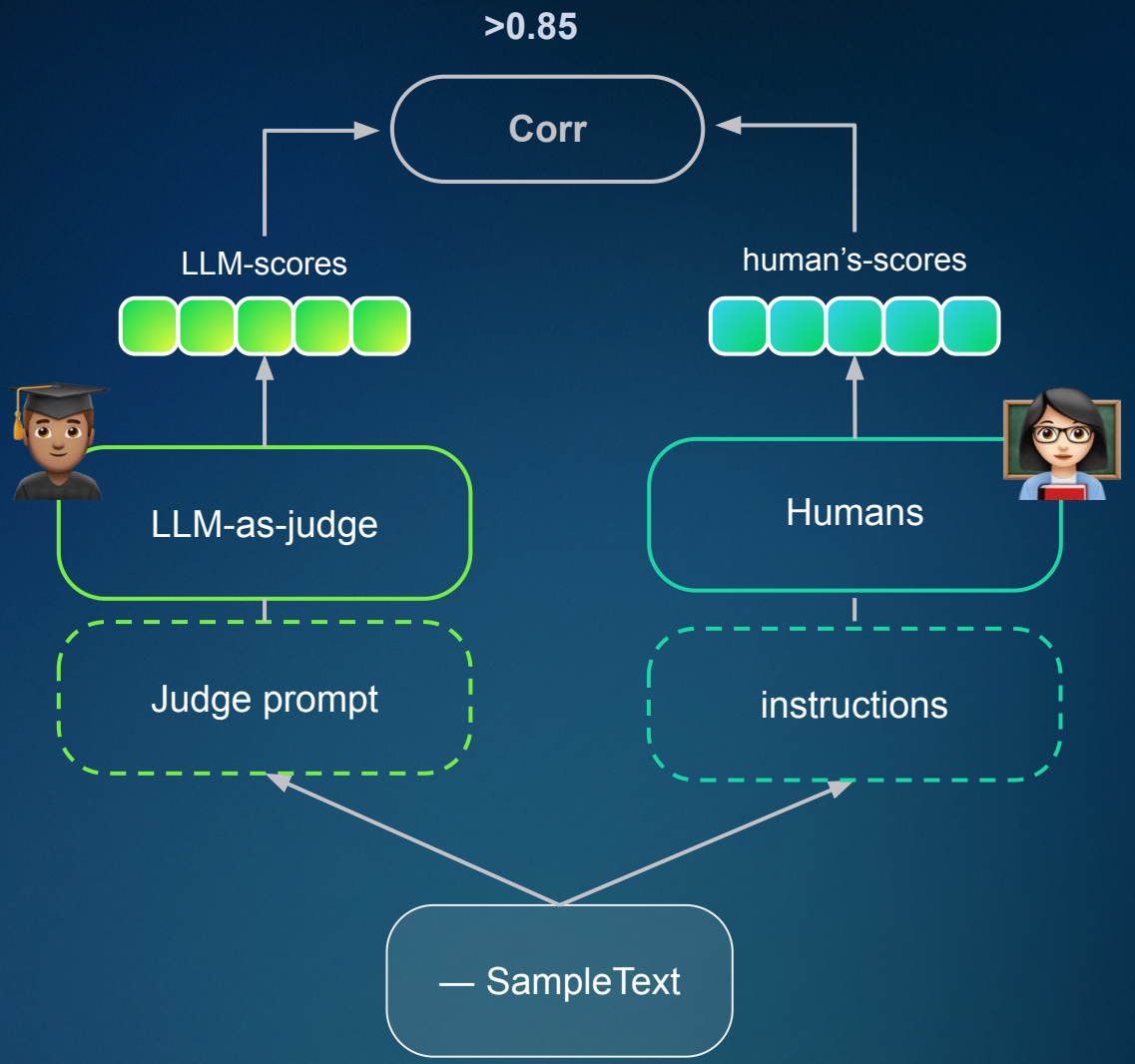
Как оценивать RAG? LLM as judge.

Для использования LLM as judge ВАЖНО помнить про ряд ограничений:

1. LLM как судьи отдают предпочтение (завышают оценку) ответам: от моделей с “родственным” поведением, определенной стилистики, длинным или наоборот коротким текстами и т.п.

1. Нельзя использовать в качестве судьи ту же модель, что и в RAG ризонинге, даже того же семейства, даже после SFT.

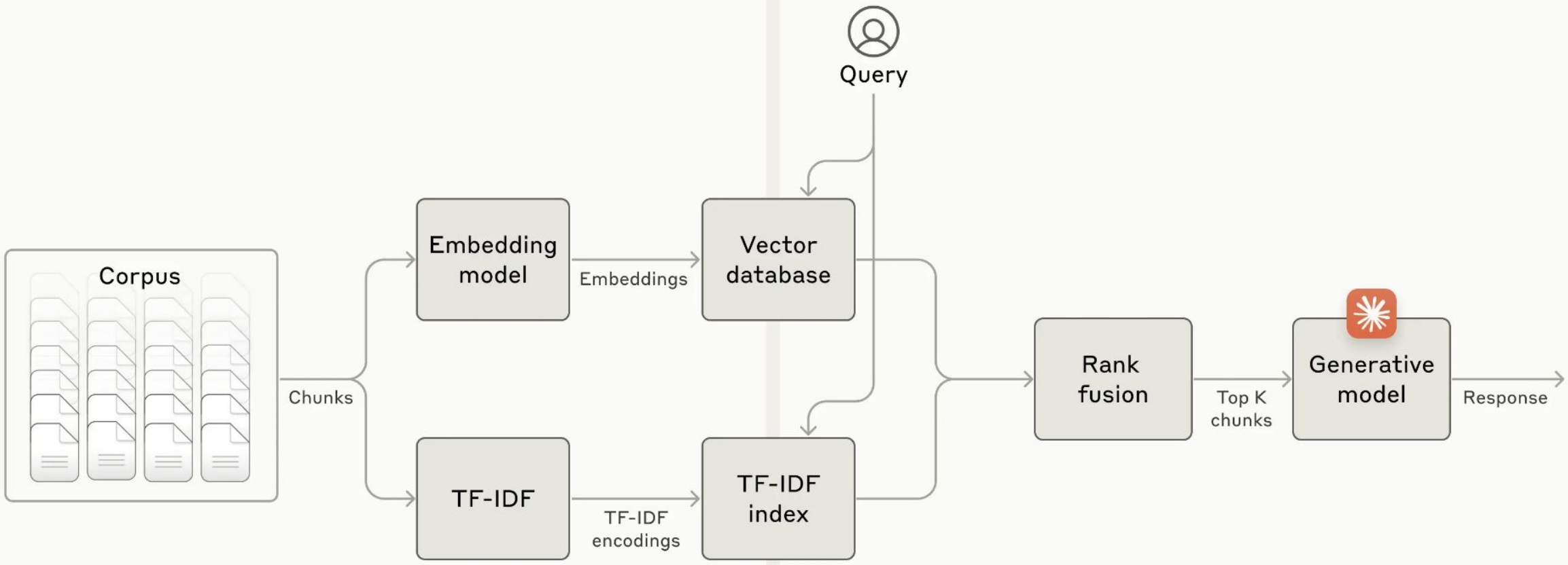
1. Инструкция в судье должна показать свою скоррелированность с людской экспертной разметкой.



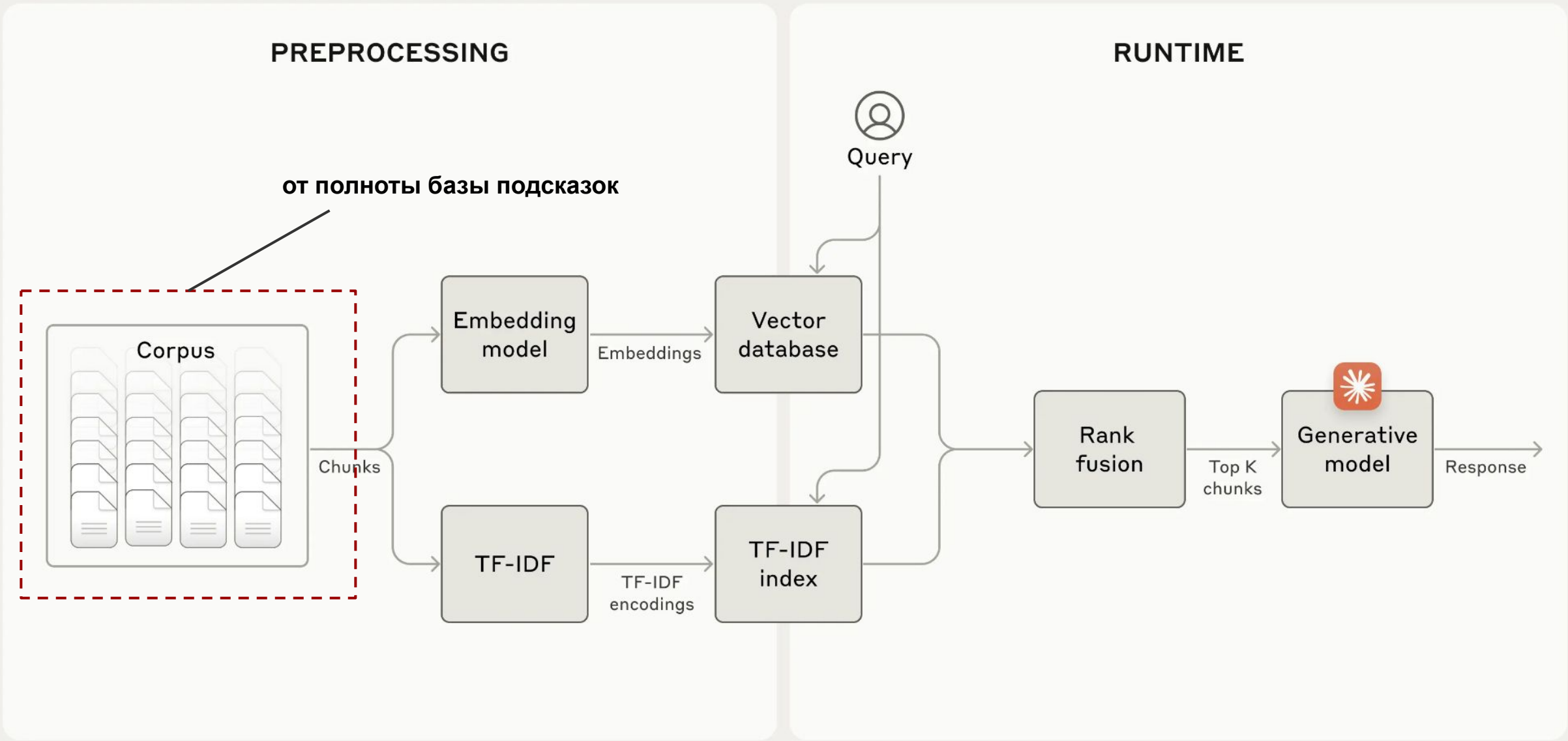
Где почитать https://habr.com/ru/articles/905728/

RAG на примере Anthropic

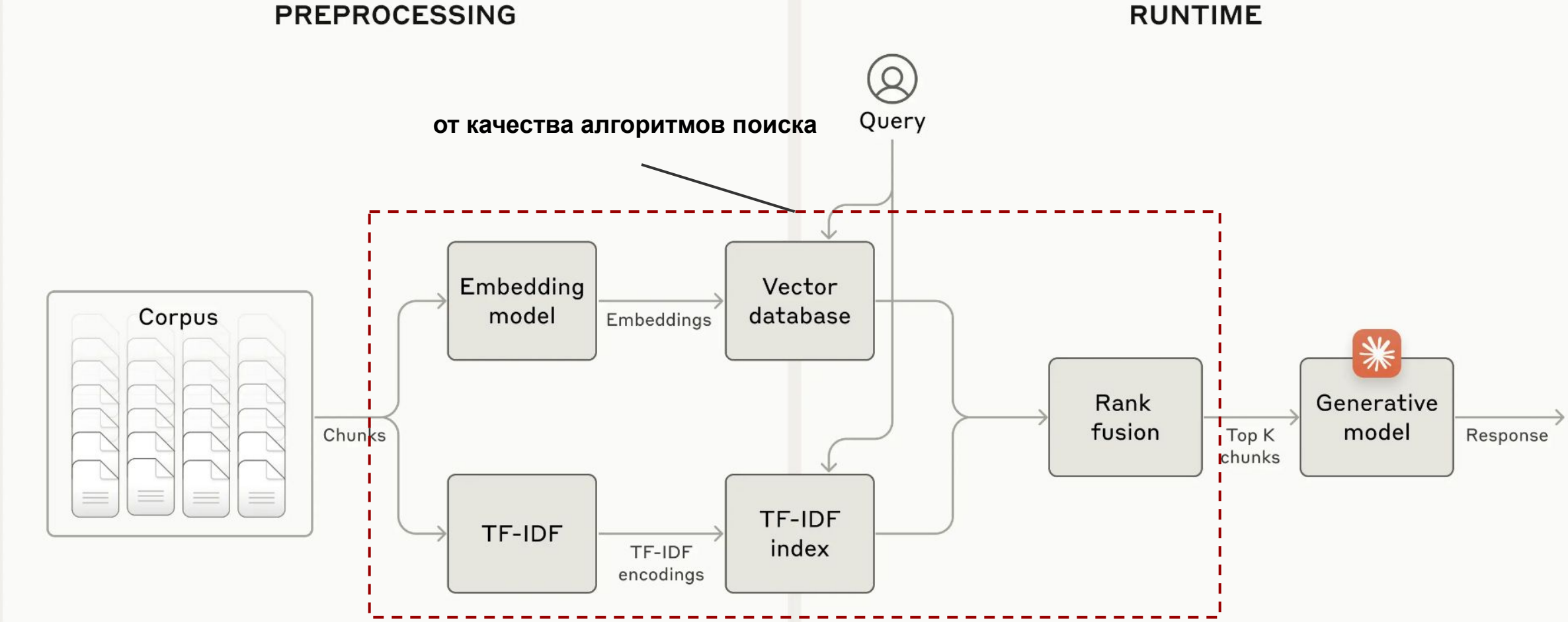
Standard RAG



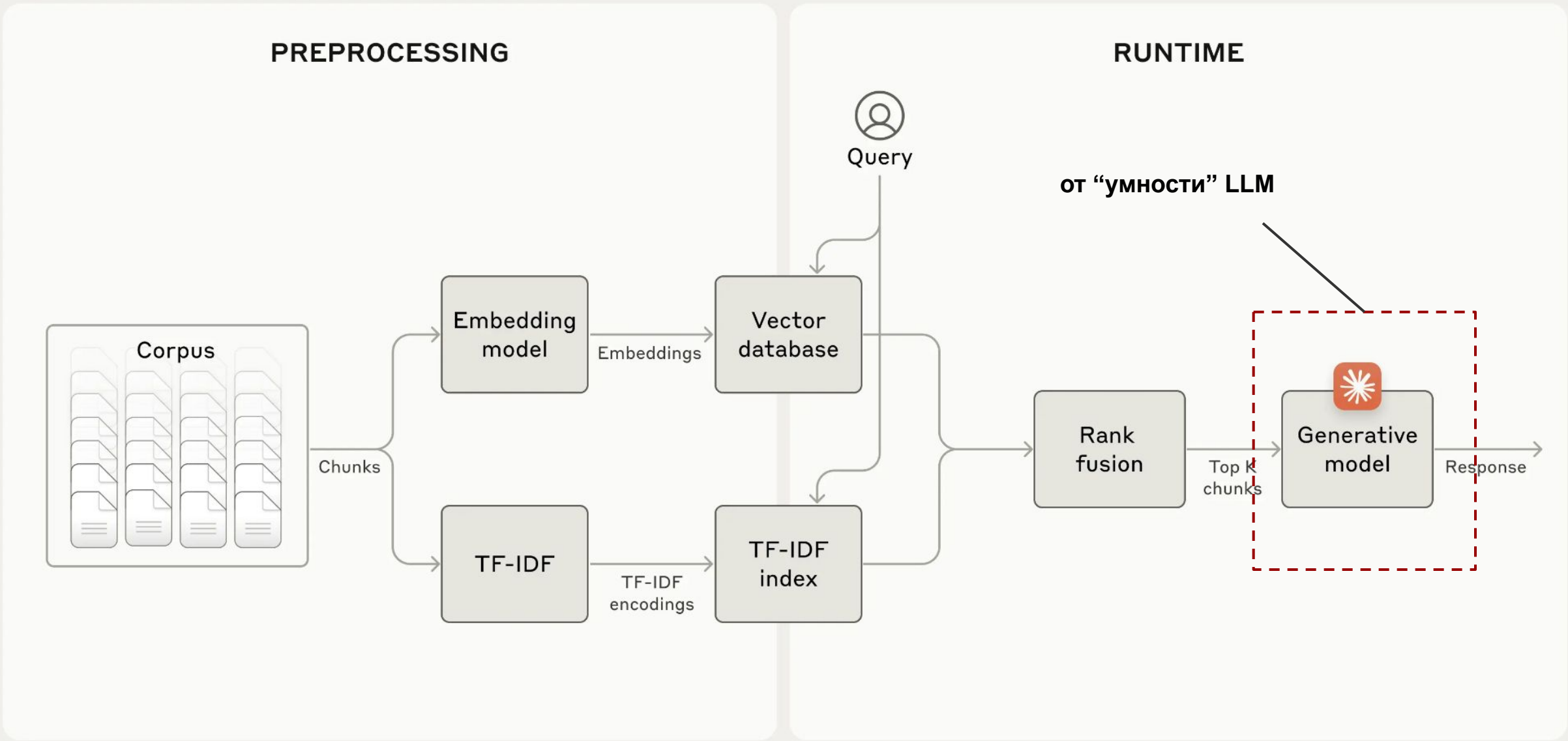
Standard RAG



Standard RAG

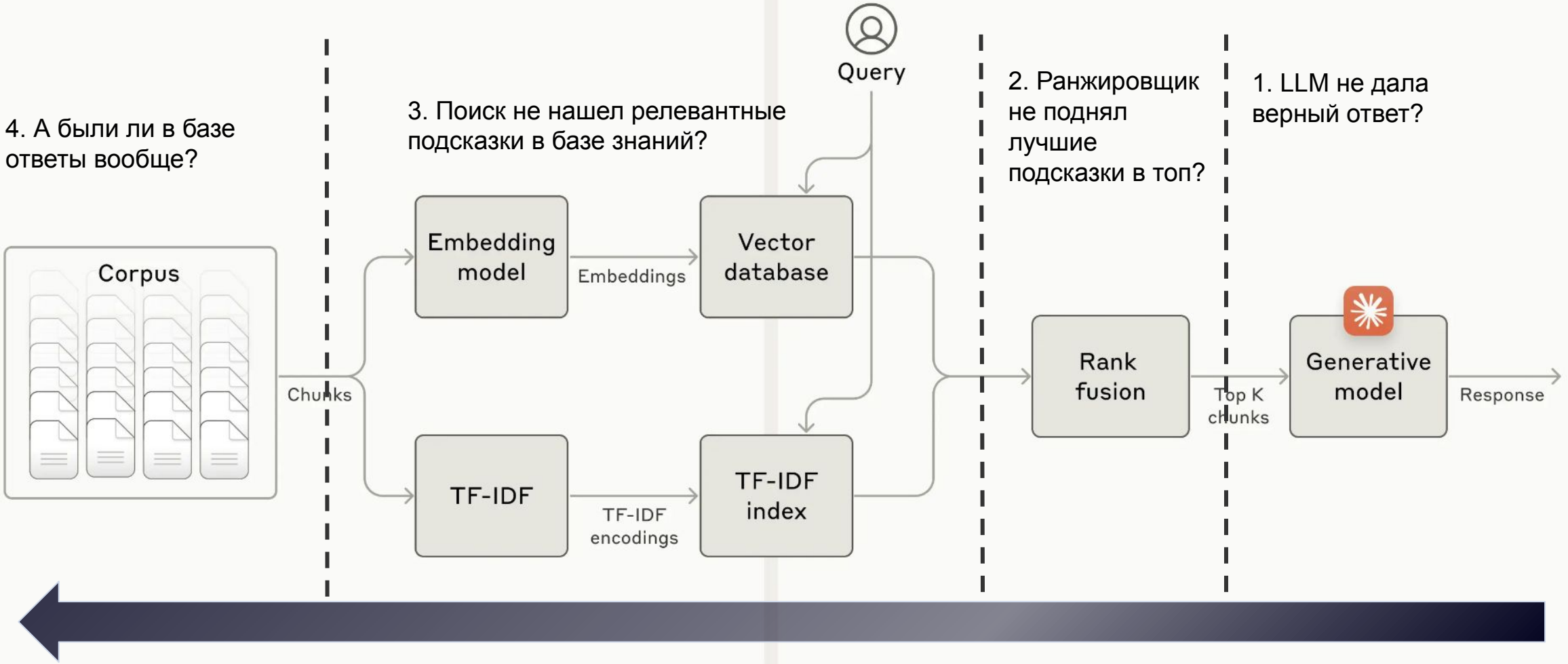


Standard RAG

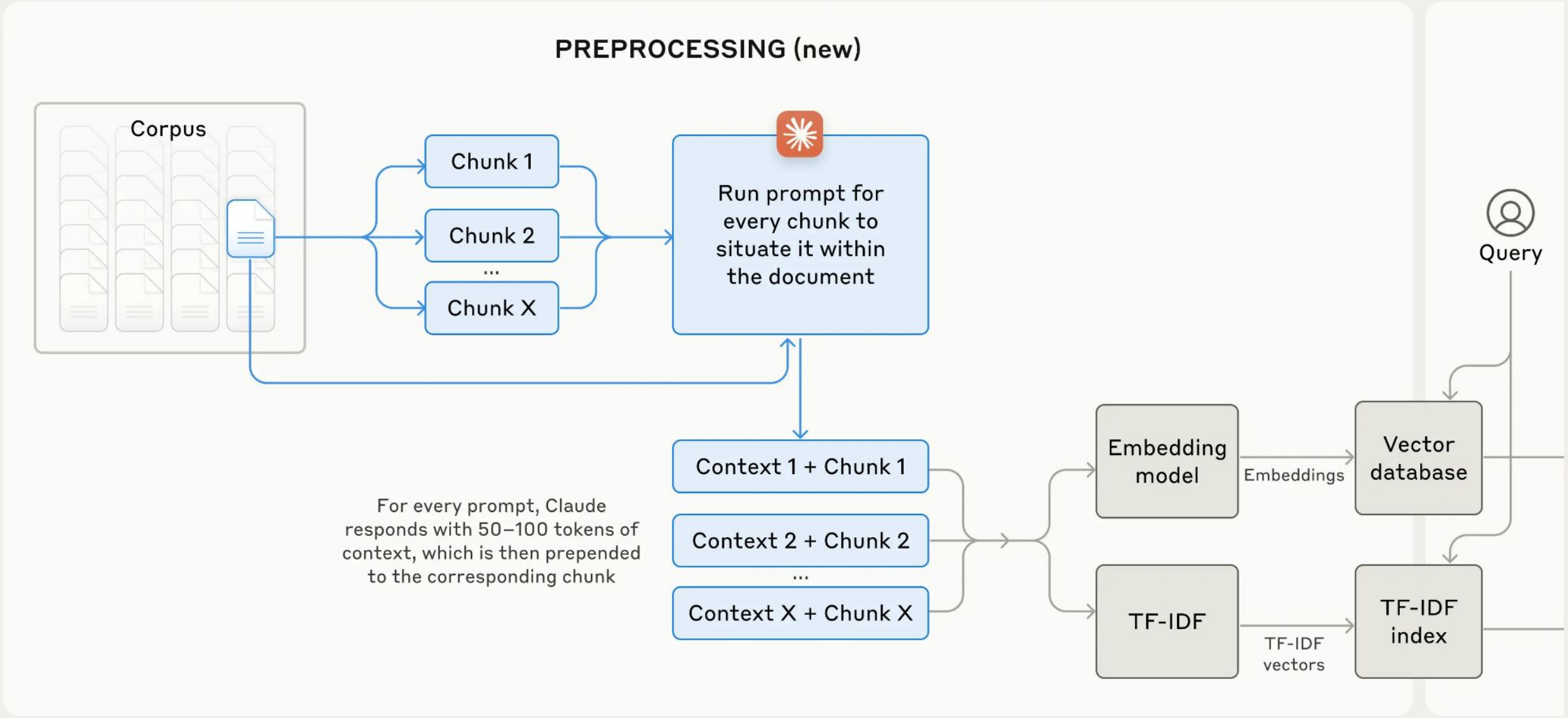


Пример метода “обратного хода” в оценке e2e систем с ИИ.

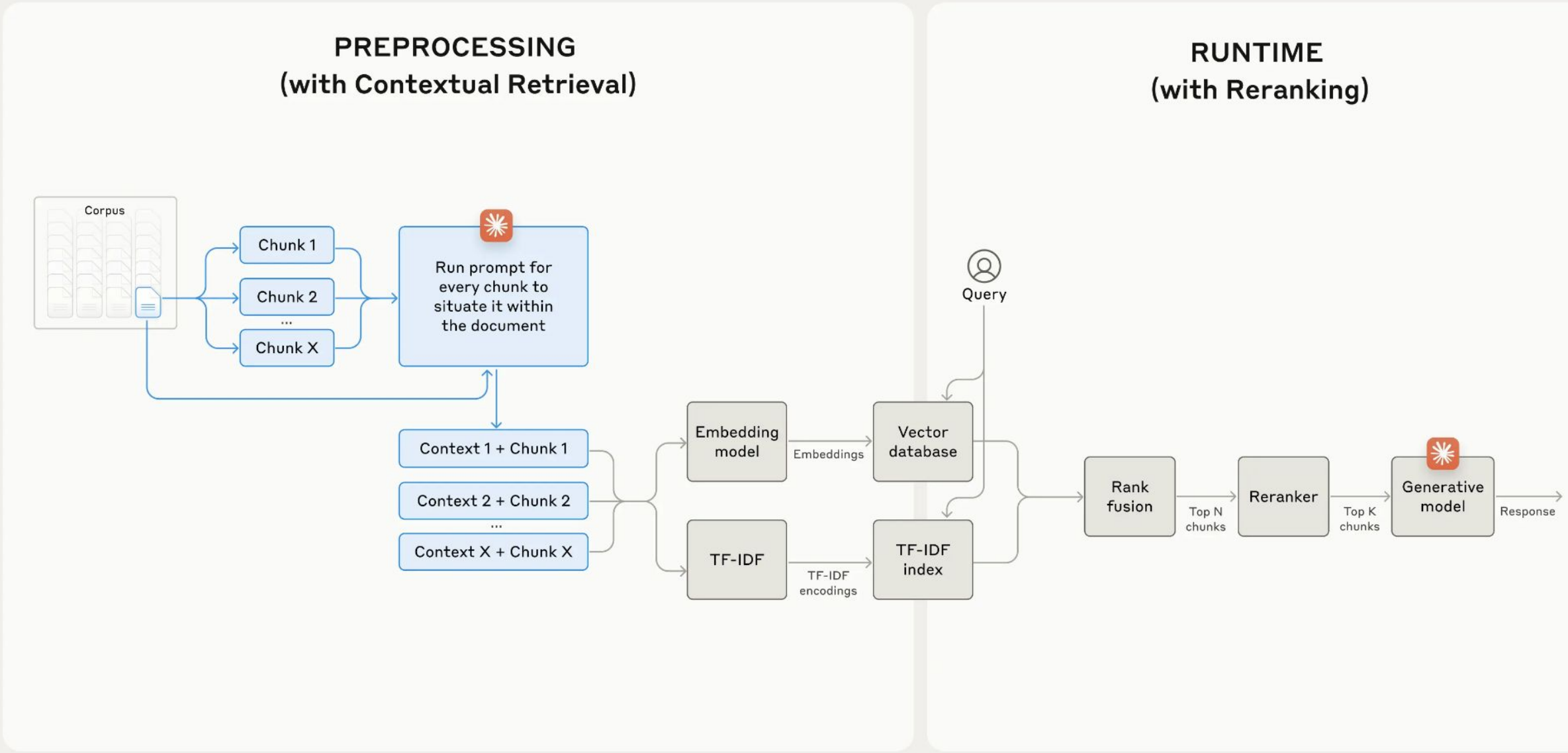
Рассмотрим алгоритм проведения оценки качества системы с RAG. Данный подход при измерении на тестовом датасете позволит разложить ошибки по полочкам, и точечно, в нужном модуле обнаружить и устранить проблему. Данный подход НЕ требует выкатки на А/Б и позволяет до А/Б предотвратить падение CSI/NPS метрик.



Contextual Retrieval Preprocessing



Combined


PREPROCESSING
(with Contextual Retrieval)
Corpus
Chunk 1
Chunk 2
...
Chunk X
Run prompt for every chunk to situate it within the document
Context 1 + Chunk 1
Context 2 + Chunk 2
...
Context X + Chunk X
Embedding model
Embeddings
Vector database
TF-IDF
TF-IDF encodings
TF-IDF index
RUNTIME
(with Reranking)
Query
Rank fusion
Top N chunks
Reranker
Top K chunks
Generative model
Response

Спасибо за внимание!

@DEALERAI

Александр Абрамов

@dealer_ai

Дополнительные материалы